

Problem Solving Strategies and Computational Approaches SCS1304

Handout 6 : Greedy Algorithms

Prasad Wimalaratne PhD

Overview

- Greedy Algorithms
 - Coin Change Problem
 - Kruskal's Algorithm
 - Prim's Algorithms

Greedy Algorithms

Prims and Kruskals Algorithms - Greedy Method

<https://www.youtube.com/watch?v=4ZIRH0eK-qQ>

- Greedy Method - Introduction

https://www.youtube.com/watch?v=ARvQcqJ_-NY&list=PLfFeAJ-vQopt_S5XlayyvDFL_mi2pGJE3

- Prim's Algorithm for Minimum Spanning Tree

<https://www.youtube.com/watch?v=ZtZaR7Ecl5Y>

Greedy Algorithms

- According to the Oxford English Dictionary, **greedy** means having an excessive desire for something, often without considering the consequences.
- In computer science, a greedy algorithm is an **algorithmic design technique** that builds a solution step by step by **choosing the locally optimal option at each stage, hoping that these local choices will lead to a globally optimal solution**



[1*mKvyFosEdJxv5oZpRMIQOw.gif](https://www.giphy.com/gifs/1*mKvyFosEdJxv5oZpRMIQOw) (480×362)

History of Greedy Algorithms

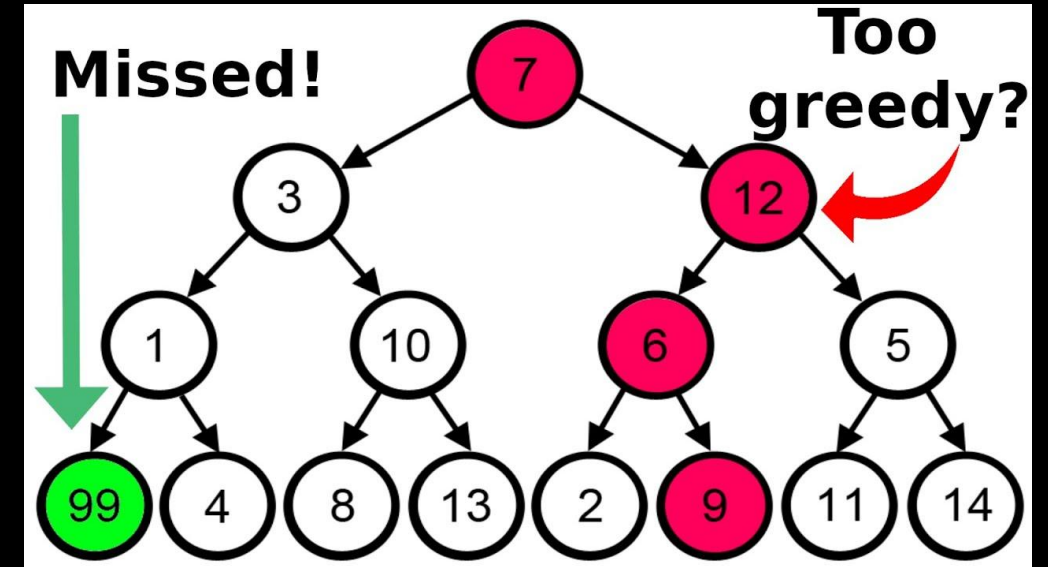
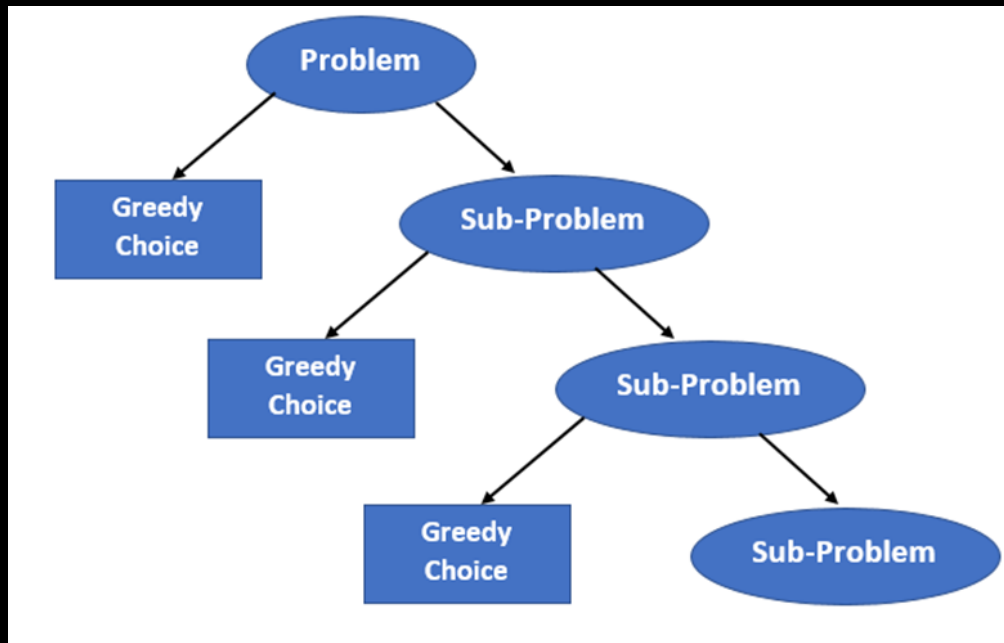
- The concept of greedy algorithms has a long **history in mathematics and problem-solving**, with similar ideas appearing in ancient times.
- Although the term **greedy algorithm** was introduced much later in computer science, the principle of making the **locally optimal choice at each step has been used for centuries**.
- One of the earliest examples is the **coin change problem**, where people needed to make change using the **fewest coins possible**.
 - A greedy strategy for this problem is to repeatedly choose the largest coin denomination that does not exceed the remaining amount until the total amount is reached.

```
While W > 0  
  pick the largest coin c that is <= W  
  W <- W - c
```

- This strategy works correctly for some currency systems, but it does not always produce the optimal solution for every set of coin denominations.

Greedy Algorithms

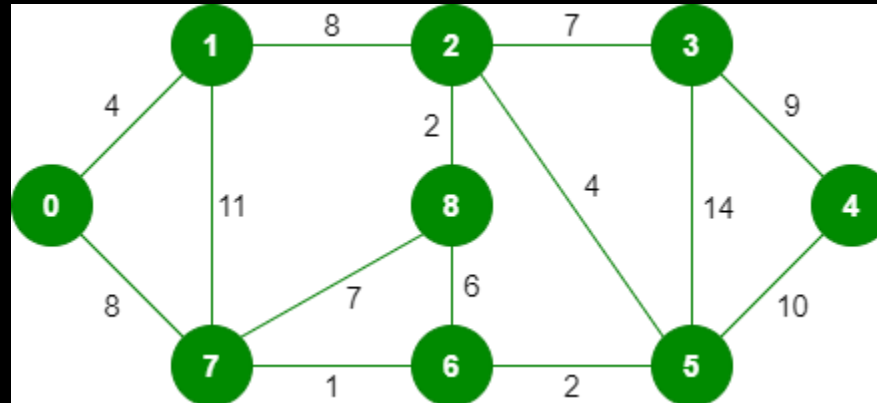
- Classic examples include **Prim's and Kruskal's minimum spanning tree algorithms, Dijkstra's shortest path algorithm.**
- Greedy algorithms are efficient but do NOT always guarantee the optimal solution for every problem.
- The term **greedy algorithm** became widely used in **computer science** during the development of algorithm design techniques in the **1960s and 1970s.**



<https://www.youtube.com/watch?v=6CKTePpUIJI>

Greedy Algorithms..

- Edsger W. Dijkstra developed the famous **shortest path algorithm** (1956–1959), which follows a **greedy strategy**.
- Robert C. **Prim** (1957) and Joseph **Kruskal** (1956) independently developed greedy algorithms for finding a **Minimum Spanning Tree (MST)**.
- Many **greedy algorithms** have been developed for solving **graph problems**, such as finding **minimum spanning trees**, **shortest paths**, **scheduling**, **data compression**, and **network optimization**.
- A **graph** is a mathematical structure consisting of **vertices (nodes)** connected by **edges (links)**.



Re Cap Asymptotic Analysis of Algorithms

Examples

1. Polynomial Time

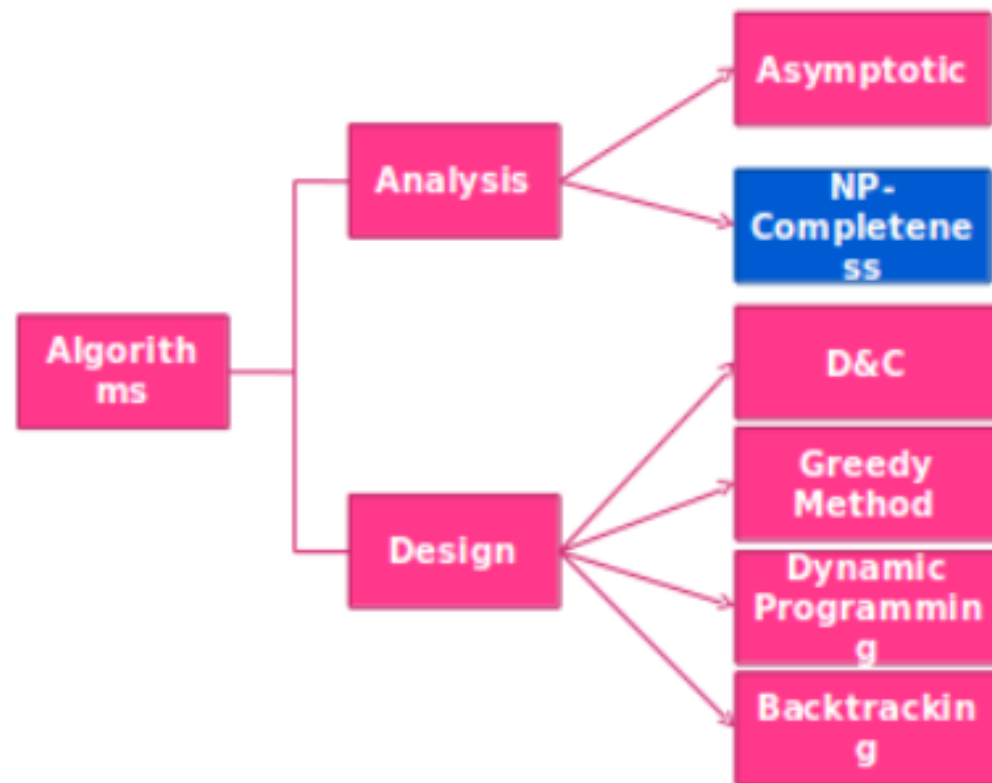
- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$

2. Non-Polynomial Time

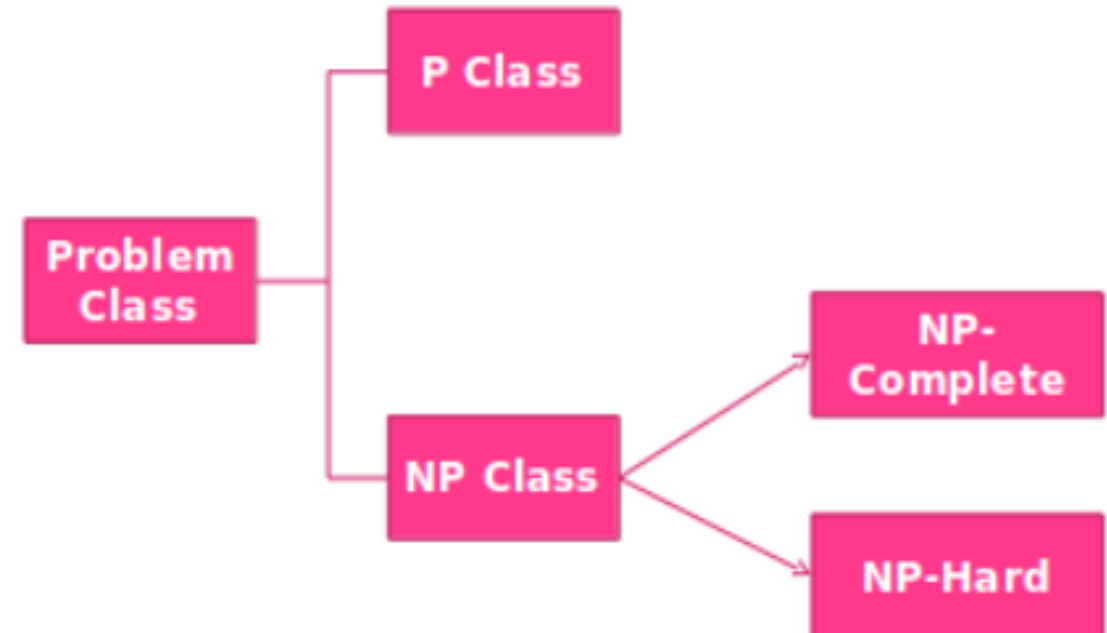
- $O(2^n)$, $O(n!)$

These algorithms become impractical as the input size grows

P and NP Class Algorithms



Problem Class



<https://gist.github.com/gambitier/49f8c6afa21f0e6c721682f72c541c8a>

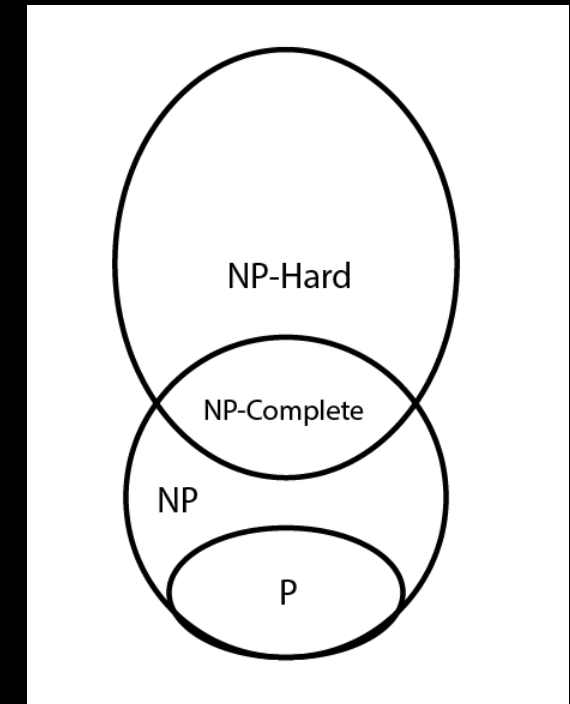
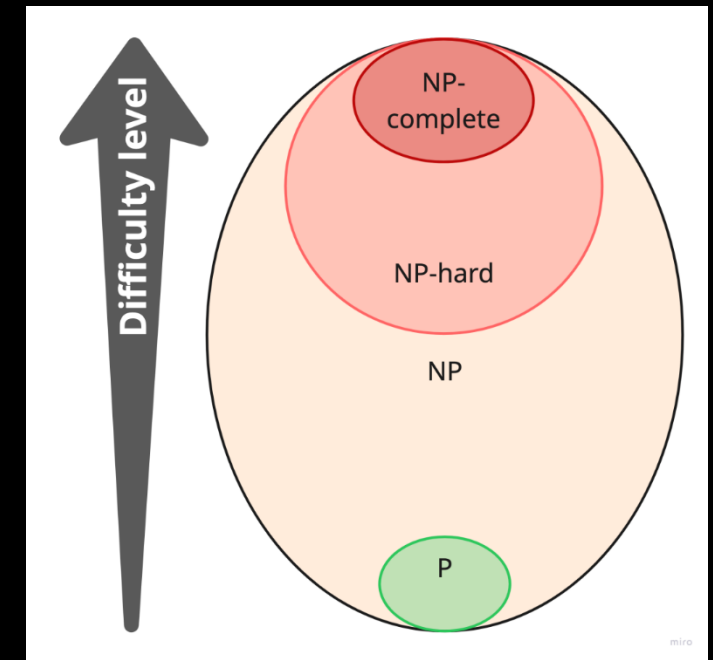
P vs. NP problem : The unsolved Mistry in CS

1. NP stands for **Nondeterministic Polynomial Time**. It is a group of problems where, if you guess the right answer, a computer can check it quickly (in polynomial time).
 - A simple analogy is a Sudoku puzzle. Finding the solution **from scratch** is **very hard**, but if someone hands you a completed board, you **can quickly verify it is correct**.
2. NP-complete problems are the hardest problems in the NP group. We can verify their solutions quickly, but NO one has found an algorithm to solve them quickly. If you find a fast way to solve *just one* NP-complete problem, you can solve *all* NP problems quickly. (i.e **if anyone finds a fast (polynomial-time) algorithm for even one NP-complete problem, then every problem in NP can also be solved fast**). This is because all NP-complete problems can be transformed ("reduced") into each other in polynomial time.
 - This famous puzzle is called the P vs. NP problem, and it remains one of the greatest unsolved mysteries in computer science

- NP is the group, and NP-complete problems are the hardest, most representative members of that group
- If P = NP turned out to be true, it would mean fast-to-verify problems are also fast-to-solve.
- This has implications for cryptography, optimization, logistics

NP-Complete

- **NP-C or NPC, NP-Complete** is a classification of problems in computer science
- **NP-complete problems are verified but not solved by a computer in a reasonable time.**
- **NP** (Non-deterministic Polynomial Time) is the **class of decision problems** whose proposed solutions **can be verified in polynomial time.**
- The **NP** stands for **non-deterministic polynomial time** and refers to how long a computer would take to verify the problem is solved.
- For example, **NP-complete problems** are important in **cryptography**, where it is important to know if **a password can be guessed given enough time trying a random combination of letters and numbers.**



NP-hard Problems

NP-hard refers to a class of computational problems that are at least as hard as the hardest problems in NP.

1. **P**: solvable in polynomial time. i.e. these problems can be solved in time $O(n^k)$ in worst-case, where k is constant.
 2. **NP**: a proposed solution (**certificate**) can be verified in polynomial time.
 - If we are somehow given a '**certificate**' of a solution we can verify the legitimacy in polynomial time. We are NOT asking for a way to find a solution, but only to verify that an alleged solution really is correct.
 3. **NP-hard**: Problems at least as hard as the hardest problems in NP; **NO known polynomial-time algorithm solves them optimally** in general.
 4. **NP-complete** = **NP-hard** $\cap$ **NP** (i.e the **hardest problems** that are still in NP)
-
- ❖ A deterministic algorithm follows one specific sequence of steps.
 - ❖ A non-deterministic algorithm is a theoretical model that can 'guess' the correct solution and then verify it in polynomial time.

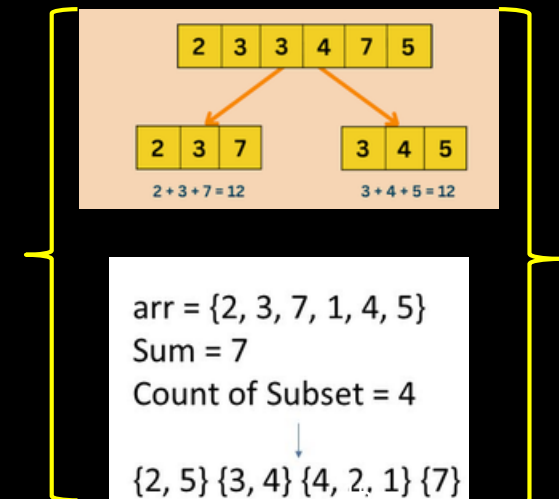
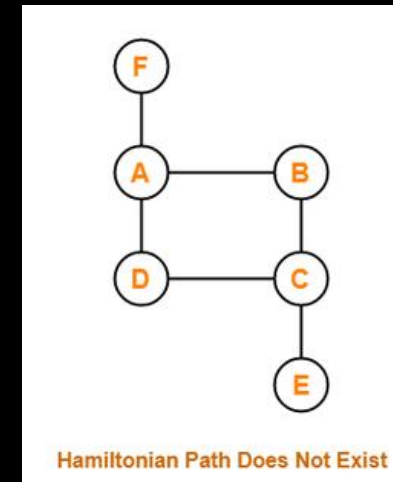
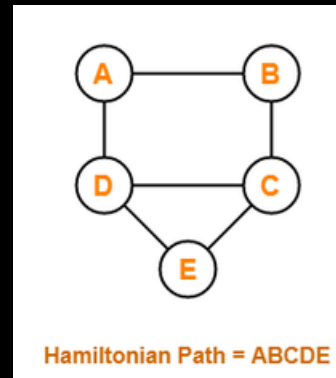
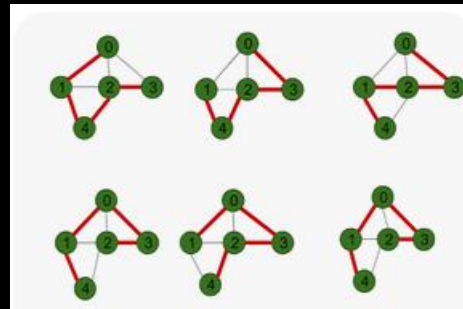
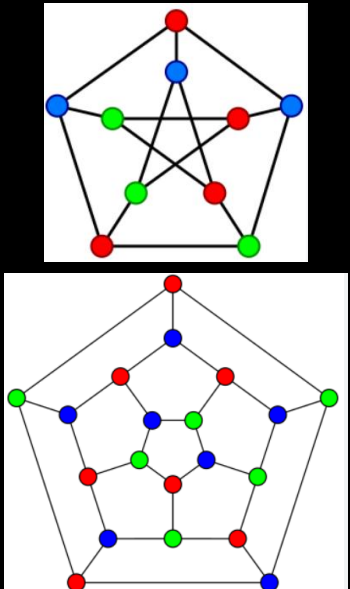
<https://gist.github.com/gambitier/49f8c6afa21f0e6c721682f72c541c8a>

P-Hard and NP-Complete Problems

<https://www.youtube.com/watch?v=e2cF8a5aAhE>

Classic examples

1. **Traveling Salesman Problem (TSP)** : find the shortest route visiting a set of cities exactly once.
2. **Knapsack Problem** : choose items to maximize value under a weight limit.
3. **Graph Coloring** : color a graph's vertices so **NO adjacent vertices share a color**, using the fewest colors.
4. **Hamiltonian Path**: find a path that **visits every vertex exactly once**.
5. **Subset Sum** : determine if a **subset of numbers sums to a target value**.



How to solve NP-hard problems in practice

Since exact solutions can be computationally infeasible for large inputs, practitioners often use:

1. **Approximation algorithms** (get close to optimal, with provable bounds)
2. **Heuristics** (genetic algorithms, simulated annealing, **greedy methods** are good solutions but NO guarantees of optimal solutions)
3. **Special-case algorithms** (exploit structure in the specific instances you care about)
4. **Exponential algorithms with pruning** (**branch-and-bound**, **backtracking** which is more suitable for **small inputs**)

Greedy algorithms

- Greedy algorithms are commonly used to solve **optimization** problems, where the **objective is to find the maximum or minimum value of a particular quantity while satisfying given constraints.**
- A solution that achieves the **best possible value of the objective function** is called an **optimal solution**.
- Greedy algorithms construct a solution by making the **locally optimal choice at each step, without reconsidering previous decisions.**

Greedy algorithms

- Solves a problem step by step by making the **best local (greedy) choice** at each stage.
- Each choice **aims to move the solution towards the best possible overall result.**
- **Simple** to implement and computationally **efficient.**
- Produces the **optimal** solution for problems that **satisfy:**
 - a. **Greedy-choice property**
 - b. **Optimal substructure**
- Does **NOT** always **guarantee** the **optimal solution.**
- For many complex problems (especially where finding the optimal solution is difficult), it **provides a good approximate (near-optimal) solution.**
- The **quality** of the solution can often be measured using an **approximation ratio, which compares it to the optimal solution.**
- Breaks a large problem into smaller subproblems and solves them independently by making the best choice at each step.

Greedy algorithms

- A **heuristic** is a practical problem-solving approach that uses experience or rules of thumb to make good decisions quickly when time, information, or computational resources are limited.
- **Heuristics do NOT always guarantee the optimal solution.**
 - Finding a Friend in a Crowd?
 - **Heuristic:** Search places where they are **most likely to be** (e.g., near the entrance or food court) **instead of searching randomly**.
 - Result: Usually faster, but not guaranteed to find them immediately.
 - Chess ?
 - **Heuristic:** Capture the opponent's **most valuable piece OR move to protect your king**.
 - Result: Often a **good move, but not necessarily the best move** in the entire game.
- Greedy heuristics **often generate good approximate solutions** in a reasonable amount of time, making them useful for large or complex problems.

How Does the Greedy Algorithm Work?

. It is similar to taking the best option available at each moment, **hoping it leads to the best overall result**, without looking ahead or reconsidering past choices.

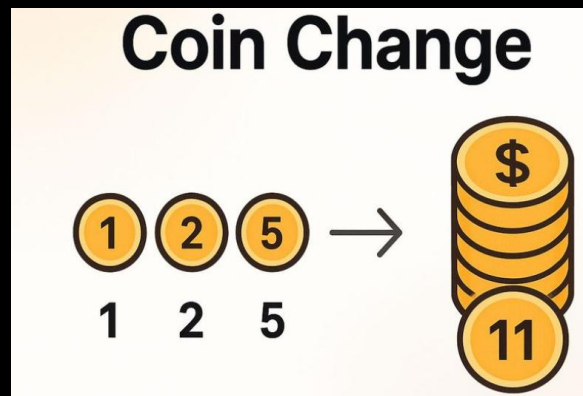
Steps:

1. Start with the **initial state of the problem**. This is the starting point from which you begin making choices.
2. Evaluate all **possible choices available from the current state**. Consider every option on the table at that moment.
3. Choose the **option that seems best right now, regardless of future consequences**. This is the **greedy** part. You take the best looking option available at this step, even if it might not be optimal in the long run.
4. **Move to the new state** resulting from your chosen option. This becomes the starting point for the next iteration.
5. **Repeat steps 2 to 4** until you reach the goal state, or until no further progress is possible. Keep making locally best choices until the problem is solved or you get stuck.

Coin change Problem

Example:

- Coin change with denominations {1, 5, 10, 25} for 30 rupees.
- Greedily pick 25, then 5. Done in 2 coins (optimal here).
- But with denominations {1, 3, 4} for 6 rupees, greedy picks 4, then 1, then 1, giving 3 coins, when 3+3 would have given 2 coins instead.
- This shows why the greedy choice is NOT always globally optimal.



Coin Change Problem

For example, with $C=\{1,2,5\}$ and $W=13$, you will pick 5, 5, 2 and 1 , and you can show that the minimum number of coins required is indeed 4 .

Given coin denominations: $C = \{1, 2, 5\}$

Amount: $W = 13$

Greedy strategy:

1. Choose 5 (remaining = 8)
2. Choose 5 (remaining = 3)
3. Choose 2 (remaining = 1)
4. Choose 1 (remaining = 0)

Solution: {5, 5, 2, 1}

```
GreedyCoinChange(C, W)
```

1. Sort the coin denominations in descending order.
2. For each coin c in C :
 while $W \geq c$:
 Select coin c
 $W = W - c$
3. If $W = 0$:
 Return the selected coins
Else:
 Return "No solution"

Coin Change Problem ...

However, **Greedy algorithm does NOT always provide an optimal solution**. For example, if $C=\{1,4,5\}$ and $W=8$, the greedy algorithm will choose coins **5,1,1,1** when there is a solution **4,4** using less coins.

Given coin denominations: $C = \{1, 4, 5\}$

Amount: $W = 8$

Greedy strategy:

1. Choose 5 (remaining = 3)
2. Choose 1 (remaining = 2)
3. Choose 1 (remaining = 1)
4. Choose 1 (remaining = 0)

Greedy solution: $\{5, 1, 1, 1\}$

Number of coins = 4

However, a better solution is: $\{4, 4\}$

Number of coins = 2 (Optimal)

```
GreedyCoinChange(C, W)
```

```
1. Sort the coin denominations in descending order.
2. For each coin c in C:
    while W ≥ c:
        Select coin c
        W = W - c
3. If W = 0:
    Return the selected coins
Else:
    Return "No solution"
```

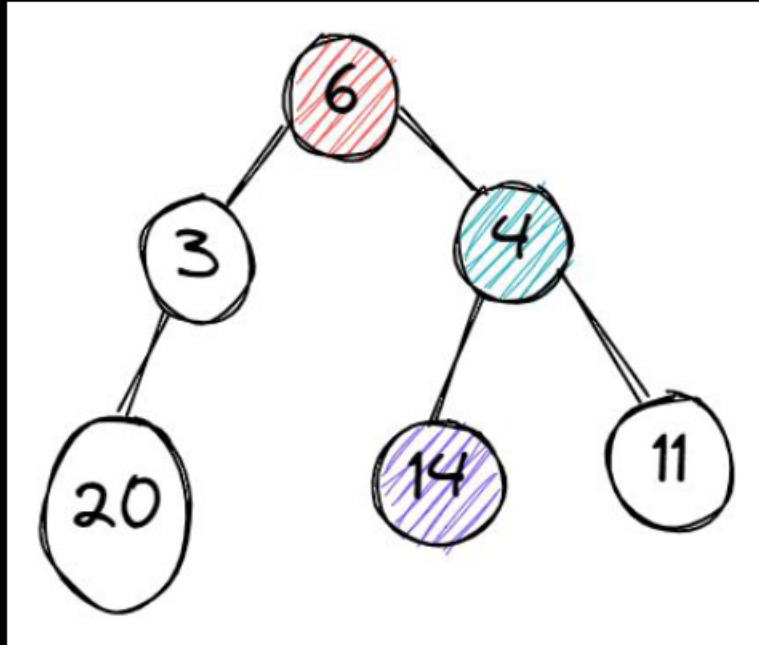
Further Example: Coin Change Problem

Let us say you have a set of coins with values {1, 2, 5, 10, 20, 50, 100} and you need to give minimum number of coin to someone change for 36 .

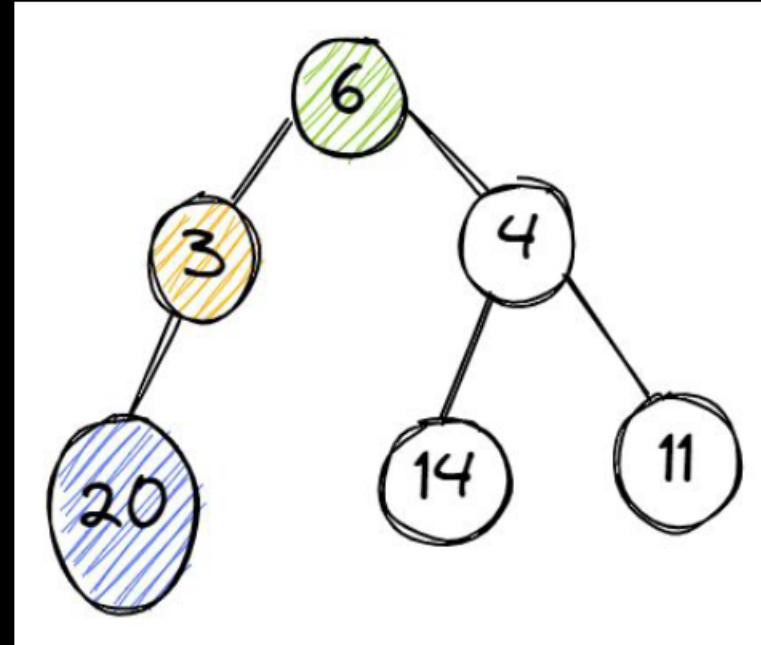
- The greedy algorithm for making change would work as follows:
 1. Start with the largest coin value that is less than or equal to the amount to be changed. In this case, the largest coin less than 36 is 20 .
 2. Subtract the largest coin value from the amount to be changed, and add the coin to the solution. In this case, subtracting 20 from 36 gives 16 , and we add a 20 coin to the solution.
 3. Repeat steps 1 and 2 until the amount to be changed becomes 0.
- Hence, using the greedy algorithm, the solution for making change for 36 would be one 20 coins, one 10 coin, one 5 coins and one 1 coin needed.

Greedy vs Not Greedy Algorithms

- Greedy Algo



Not Greedy Algo



<https://www.freecodecamp.org/news/greedy-algorithms/>

Greedy vs Not Greedy Algorithms

- A greedy algorithm is an algorithm that makes the best local choice at each step according to a specific criterion, **without reconsidering previous choices or looking ahead to future consequences.**
- Before selecting a choice, the **algorithm typically checks that it is feasible** (i.e., it does not violate the problem's constraints).
- The correctness of a greedy algorithm depends on whether the problem satisfies the **greedy-choice property and optimal substructure.**
- If these properties **do not hold, the greedy algorithm may not produce the optimal solution.**
- NOT all optimization problems can be solved using a greedy approach.
- Some problems require techniques such as **dynamic programming, backtracking, or branch and bound** to guarantee an optimal solution.

Coin Change Problem: DP vs Greedy Algorithms

Coin Change Problem can be solved using both Dynamic Programming and Greedy Algorithms. However,

- a) Dynamic Programming always produces the optimal solution (if one exists), while
- b) Greedy Algorithm does NOT always produce the optimal solution

Greedy-choice Property

- A problem has the greedy choice property if a globally optimal solution can be reached by making a locally optimal (greedy) choice at each step without reconsidering previous choices AND without needing to explore all possible options.
- A globally optimal solution can be reached by repeatedly making the locally optimal (greedy) choice.

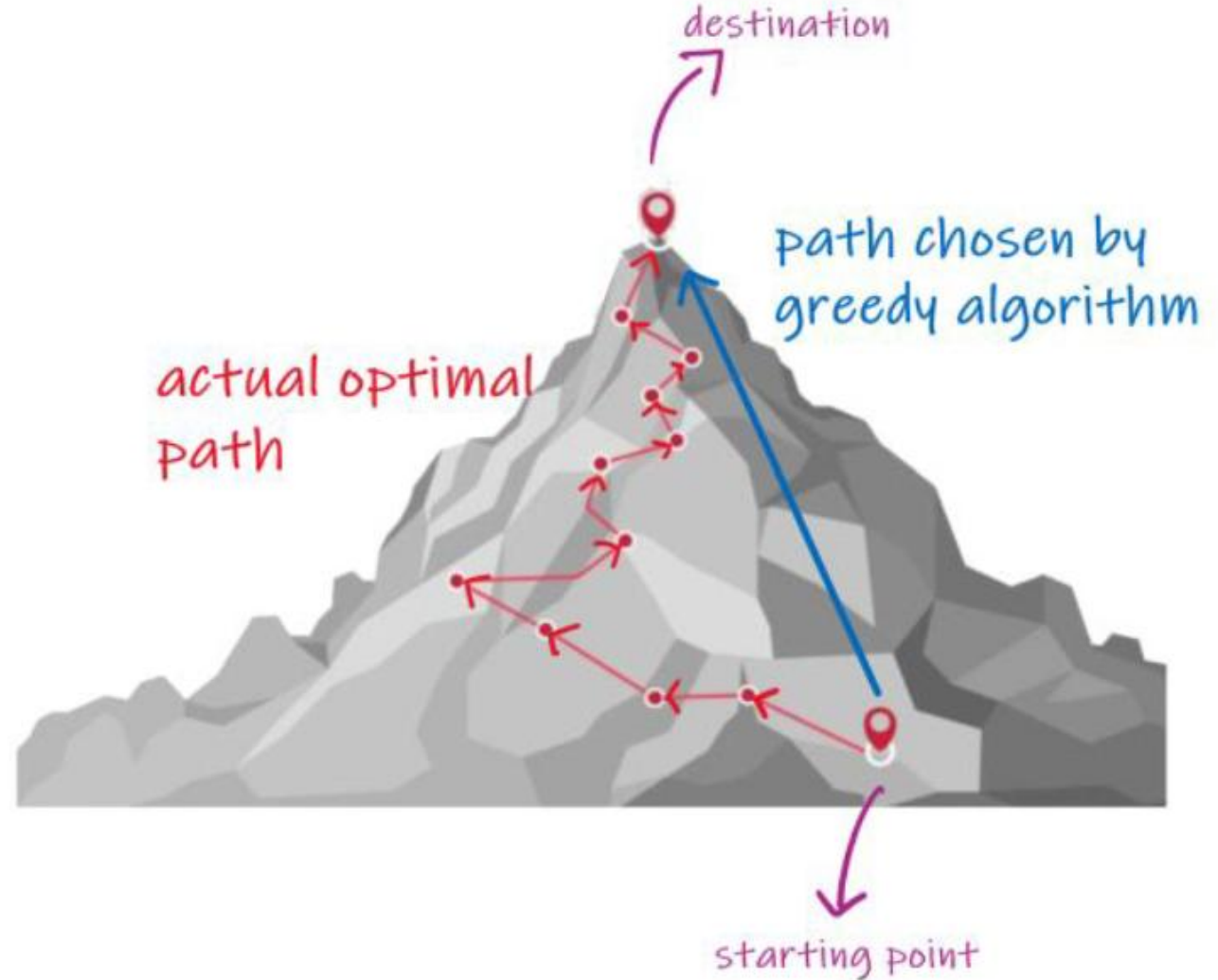
Analogy

Think of it like climbing to reach the highest point:

1. Greedy choice property :

Taking the biggest step available right now always keeps me on the path to the top (NO need to backtrack).

2. Optimal substructure : Once I am partway up, the best route from *here* to the top is independent of how I got here.



The Core Rule

1. Greedy algorithms do **NOT always guarantee** an optimal solution.
2. They **guarantee** a globally optimal **solution only when both properties hold simultaneously**

Greedy Choice Property + Optimal Substructure => Guaranteed Global Optimum

Characteristics of a Greedy Algorithm

- **Aims to find an optimal solution:**
 - The solution may involve finding a maximum or minimum value (e.g., maximum profit, minimum cost).
 - However, a greedy algorithm produces a guaranteed optimal solution only when the problem satisfies the greedy-choice property.
- **Fast and efficient:**
 - Greedy algorithms are usually simple and efficient, often having time complexities such as $O(n)$ or $O(n \log n)$, making them suitable for large-scale problems
- **No backtracking or repetition:**
 - Once a choice is made, it is usually not changed or reconsidered. The algorithm proceeds forward by making a sequence of decisions.
- **Simple and easy to implement:**
 - Greedy algorithms are generally straightforward because they avoid complex decision-making and do not explore all possible solutions.

Greedy Algorithm Example: Activity Selection Problem

- This problem contains a set of activities or tasks that need to be completed.
- Each one has a start and finish time.
- The algorithm finds the maximum number of activities that can be done in a given time without them overlapping.

Starting Time	Finish Time	Task
2	5	Homework
6	10	Presentation
4	8	Term Paper
10	12	Volleyball practice
13	14	Biology lecture
7	15	Hangout

Greedy Algorithm Examples: Activity Selection Problem

Always choose the activity that finishes first, leaving as much time as possible for the remaining activities.

This greedy choice produces the optimal solution

1. Sort all activities in ascending order of their finish times.
2. Select the **first activity** (it finishes earliest) and **add it to the selected list**.
3. For each remaining activity:
 Compare its **start time with the finish time of the last selected activity**.
 If **Start Time $\geq$ Finish Time**, then
 select the activity
 Else
 skip it
1. Repeat until all activities have been checked.

The selected list contains the **maximum number of non-overlapping activities**.

Activity Selection Problem

Starting Time	Finish Time	Task
2	5	Homework
4	8	Term Paper
6	10	Presentation
10	12	Volleyball practice
13	14	Biology lecture
7	15	Hangout

- The first step is to sort the finish time in ascending order and arrange the activities with respect to the result.

final result is the list of selected activities that we can do without time overlapping: **{Homework, Presentation, Volleyball practice, Biology lecture}**.

```

data = {
    "start_time": [2 , 6 , 4 , 10 , 13 , 7],
    "finish_time": [5 , 10 , 8 , 12 , 14 , 15],
    "activity": ["Homework" , "Presentation" , "Term paper" , "Volleyball practice" , "Biolog
}

selected_activity =[]
start_position = 0
# sorting the items in ascending order with respect to finish time
tem = 0
for i in range(0 , len(data['finish_time'])):
    for j in range(0 , len(data['finish_time'])):
        if data['finish_time'][i] < data['finish_time'][j]:
            tem = data['activity'][i] , data['finish_time'][i] , data['start_time'][i]
            data['activity'][i] , data['finish_time'][i] , data['start_time'][i] = data['act
            data['activity'][j] , data['finish_time'][j] , data['start_time'][j] = tem

# by default, the first activity is inserted in the list of activities to be selected.

selected_activity.append(data['activity'][start_position])
for pos in range(len(data['finish_time'])):
    if data['start_time'][pos] >= data['finish_time'][start_position]:
        selected_activity.append(data['activity'][pos])
        start_position = pos

print(f"The student can work on the following activities: {selected_activity}")
# Results
# The student can work on the following activities: ['Homework', 'Presentation', 'Volleybal

```

Philosophical Difference between DP and Greedy Algorithms

DP : *What is the **best way to solve this, considering all the ways smaller pieces could combine?***

exhaustive but smart (via memoization/tabulation, **avoids redundant** work).

Greedy : *What is the **best thing to do right now?***

fast but risky (assumes today's best choice won't hurt tomorrow).

Property	Dynamic Programming	Greedy Approach
Core Principle	Principle of Optimality , optimal solution built from optimal sub-solutions	Best local/feasible choice at each step
Decision Making	Considers <i>all choices, evaluates subproblems</i> , then picks best combination	Makes <i>one</i> choice per step, commits immediately, never reconsiders
Subproblems Explored	Solves overlapping subproblems, often multiple times each without memoization (or once, with it)	Solves <i>one path only, no exploration of alternatives</i>
Backtracking	Not needed (but implicitly considers alternate paths via recurrence)	Never backtracks , decisions are final

Property	Dynamic Programming	Greedy Approach
Guarantee of Optimality	Always optimal, if principle of optimality holds	Optimal only if greedy-choice property + optimal substructure both hold
Time Complexity	Usually higher (polynomial, often $O(n^2)$)	Usually lower, often $O(n \log n)$ or $O(n)$
Space Complexity	Higher, needs table/memo to store subproblem results	Lower, no need to store subproblem results
Approach Style	Bottom-up (tabulation) or top-down (memoization)	Top-down only, single forward pass
When to Use	When local choices <i>can not</i> be trusted to lead to global optimum	When problem structure guarantees local choice = global optimum

Common Applications of Greedy Algorithm

01

Fit algorithm in memory management

05

Fractional knapsack problem

02

Scheduling algorithms

06

Minimum spanning trees

03

Bin packing problem

07

Dijkstra shortest path algorithm

04

Travelling salesman problem

08

Egyptian fraction

Applications of Greedy Algorithms

1. Fit algorithm in memory management

- Fit memory management algorithms are techniques for allocating the memory blocks to the processes in an operating system.

2. Scheduling algorithms

- A typical process in a computer has input time, output time, and CPU time. These algorithms determine which process will use the CPU for execution and which processes will be on hold. These algorithms make sure that the CPU utilization is for maximum time and process execution takes place in minimum time.

Applications of Greedy Algorithms ...

3. Bin packing problem

- We need to pack items of different volumes into a **finite number of bins of fixed volume each** in such a way that we use the **minimum number of bins**. Placing data on multiple disks and **loading containers like trucks** are some real-life examples of this algorithm.

4. Travelling salesman problem

- Given a set of cities and distance between every pair of the city, the objective of Travelling salesman problem is to find the **shortest possible path to visit each city exactly once and return to the starting point**.

5. Fractional knapsack problem

- Imagine you have different items with their weights and their values. Our objective is to determine the subset of items that we should include in a collection so that their total weight is less than or equal to a given limit and their value is maximum. The fractional knapsack problem is an optimization problem.

Applications of Greedy Algorithms...

6. Minimum spanning trees

- Minimum spanning tree is a subset of the edges from starting to end in a weighted and undirected graph. MST **connects all vertices without any cycle and the total weight** of all the edges included is **minimum**. E.g [Kruskal's](#) and Prim's .

7. Dijkstra shortest path algorithm

- [Dijkstra's Algo](#) finds the shortest path between two vertices in a graph such that the **sum of weights between the edges is minimum**.

8. Egyptian fraction

- We can **represent every positive fraction** as a **sum of unique unit fractions**.
 - For example, [the Egyptian fraction](#) representation of $47/154$ is $1/7 + 1/11 + 1/14$.

Egyptian Fraction Representation of $2/3$ is $1/2 + 1/6$

Egyptian Fraction Representation of $6/14$ is $1/3 + 1/11 + 1/231$

Egyptian Fraction Representation of $12/13$ is $1/2 + 1/3 + 1/12 + 1/156$

Applications of Greedy Algorithms...

9. Coin Change Problem

- The greedy algorithm for [Coin Change Problem](#) can be used to make change for a given amount with the minimum number of coins, by always choosing the coin with the highest value that is less than the remaining amount to be changed.

6. Huffman Coding

- [Huffman Coding](#) is a technique of compressing data to reduce its size without losing any of the details. It was first developed by David Huffman. The greedy algorithm can be used to generate a prefix-free code for data compression, by constructing a binary tree in a way that the frequency of each character is taken into consideration.

Types of Knapsack Problems

1. The **Greedy Knapsack (Fractional)** problem allows items to be broken into smaller pieces, meaning a thief can **take a fraction of an item** to fill the bag. (Assumption: one copy of each item)
2. In contrast, **the 0/1 Knapsack** problem does not allow items to be divided; each item must be taken **entirely or left behind**. (It is assumed that there is exactly one copy of each item. i.e 0 = do not take the item, 1 = take the item once.)
 - ****Note** The 0/1 Knapsack problem is *NOT* typically solved optimally by a greedy algorithm. Greedy algorithms **may find** the optimal solution for **some specific instances, but not for all**. However **Dynamic Programming guarantees** the optimal solution.



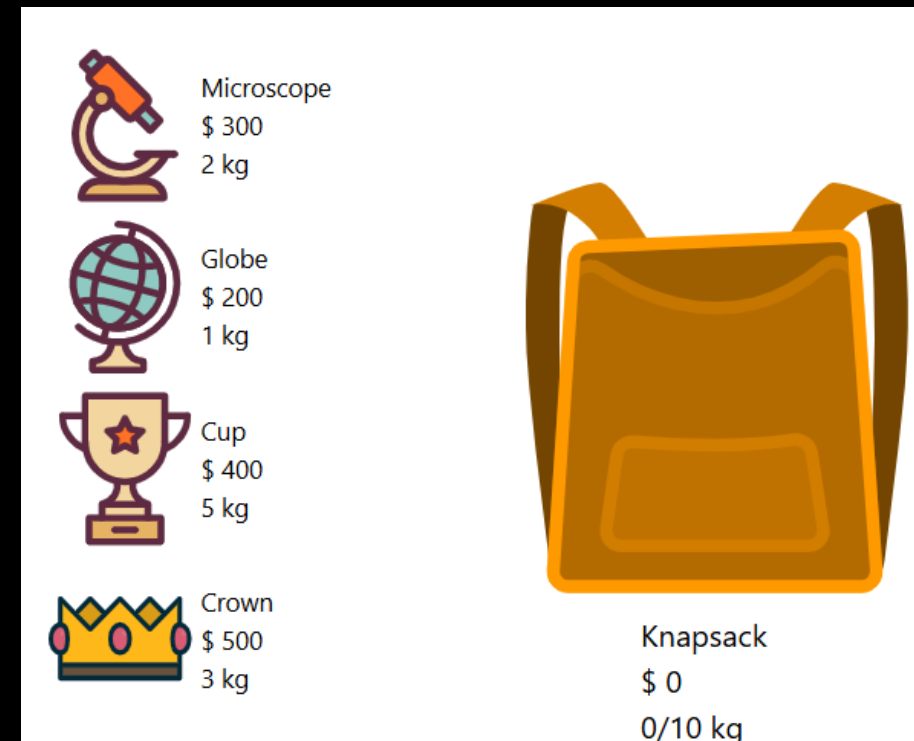
0/1 Knapsack Problem

Rules:

1. Every item has a **weight and value**.
2. Your **knapsack** has a **weight limit**.
3. Choose which items you want to bring with you in the knapsack.
4. You can **either take an item or not**, you can not take half of an item for example.

Goal:

- **Maximize the total value** of the items in the knapsack.



Fractional Knapsack

Rules

1. Every item has a **weight and a value**.
2. Your knapsack has a **maximum weight (capacity)**.
3. You can take an **entire item or only a fraction** of an item.
4. If there is not enough space for a whole item, you can take the required fraction of it.

Goal

- **Maximize the total value of the items** placed in the knapsack without exceeding the weight limit.



Advantage of Greedy Approach?

1. **Simplicity**: Greedy algorithms are generally easier to describe, understand, and implement than dynamic programming or backtracking solutions.
2. **Efficiency** : They typically run in less time and use less memory, since they **make one pass through the data, choosing the locally best option at each step** without revisiting past decisions.
3. **Good enough for many problems** : For problems that have the *greedy-choice property* (a locally optimal choice leads to a globally optimal solution), the **Greedy Approach guarantees the optimal answer**. Examples include Kruskal's/Prim's algorithms for minimum spanning trees, Huffman coding, and the activity selection problem.

Advantage of Greedy Approach?...

4. **Useful as a first approximation:** Even when it does not guarantee optimality, a greedy algorithm often produces a reasonably good solution quickly, which can be useful when an approximate answer is acceptable or as a starting point before applying a more rigorous method.
5. **Building block for other techniques:** Greedy strategies often inform or combine with other approaches (e.g., as a heuristic inside branch-and-bound or as a warm-start for optimization).

Disadvantages of the Greedy Approach

- 1. Not always optimal.** Greedy algorithms prioritize local optima over global optima, which can lead to suboptimal solutions in some cases (e.g., 0/1 Knapsack, as we saw earlier).
- 2. Difficult to prove optimality.** Proving that a greedy algorithm actually produces the optimal solution can be challenging, often requiring careful mathematical analysis (e.g., proving the greedy-choice property and optimal substructure hold).
- 3. Sensitive to input order.** The order in which input data is processed can affect the solution a greedy algorithm generates, sometimes leading to different (and not necessarily optimal) outcomes.
- 4. Limited applicability.** Greedy algorithms are not suitable for all problems and may fail on problems with complex constraints or interdependencies between choices.

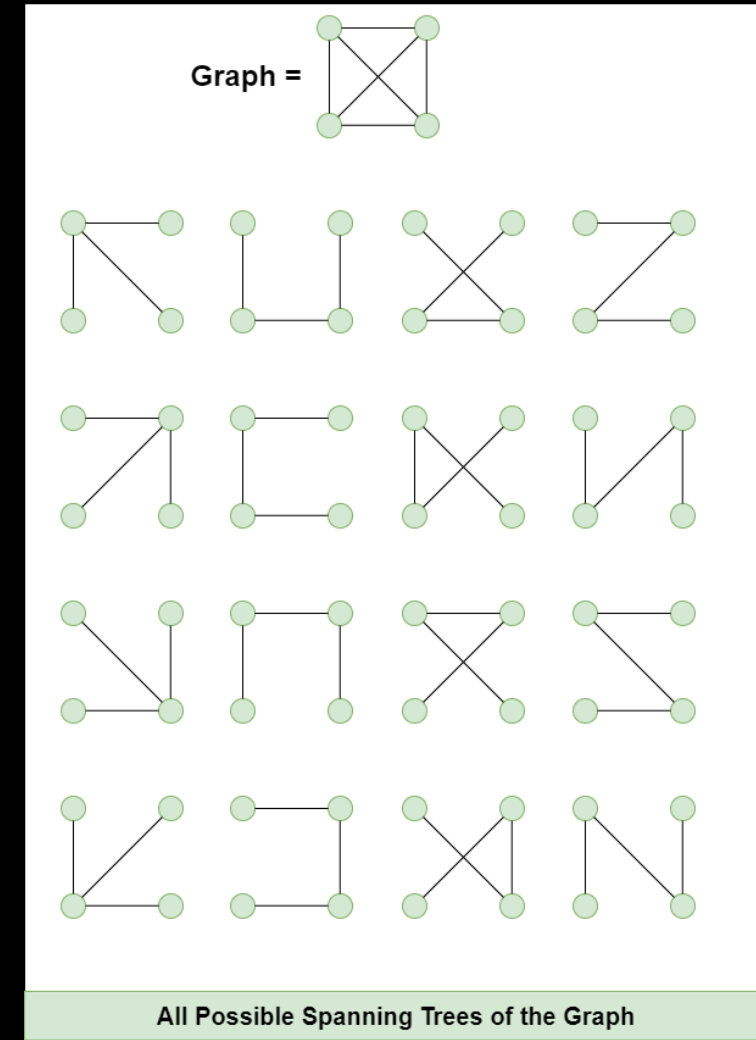
Spanning Tree

A spanning tree is defined as a tree like subgraph of a connected, undirected graph that includes ALL the vertices of the graph.

In other words, it is a subset of the edges of the graph that forms a tree (acyclic) where every node of the graph is part of the tree.

Key properties:

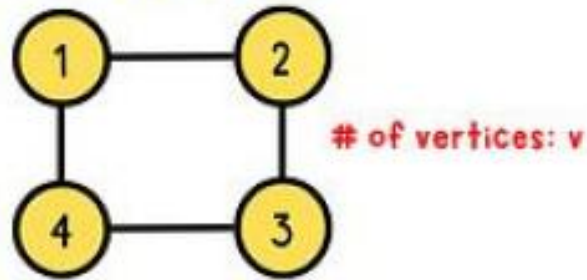
1. **Connected**. Every vertex of the original graph is reachable in the spanning tree.
 2. **Acyclic**. It contains no cycles, it is a tree, not just any subgraph.
- **Minimal edges**. A spanning tree with V vertices always has exactly $V - 1$ edges, the minimum number needed to keep the graph connected.



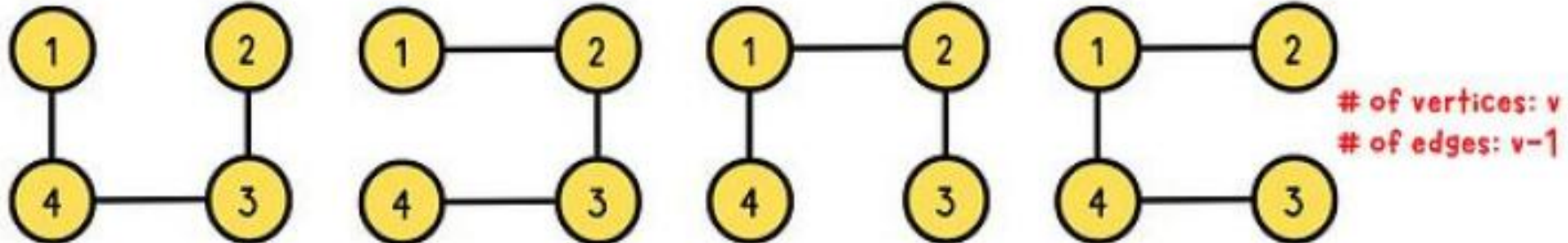
Spanning Trees

Spanning Tree

undirected graph



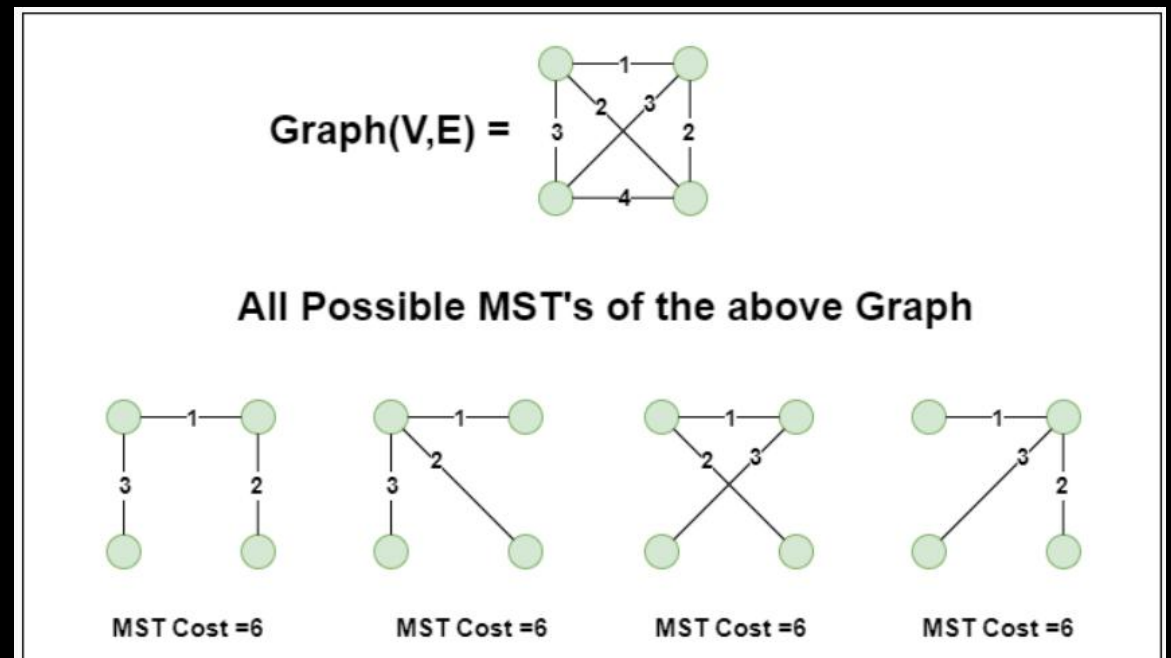
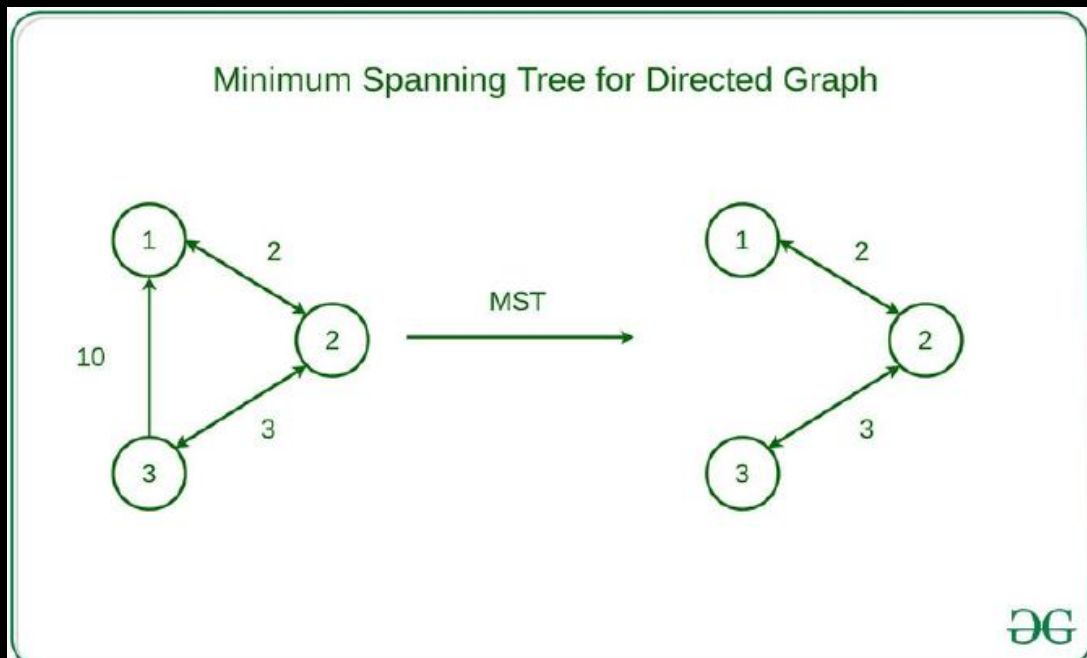
spanning trees



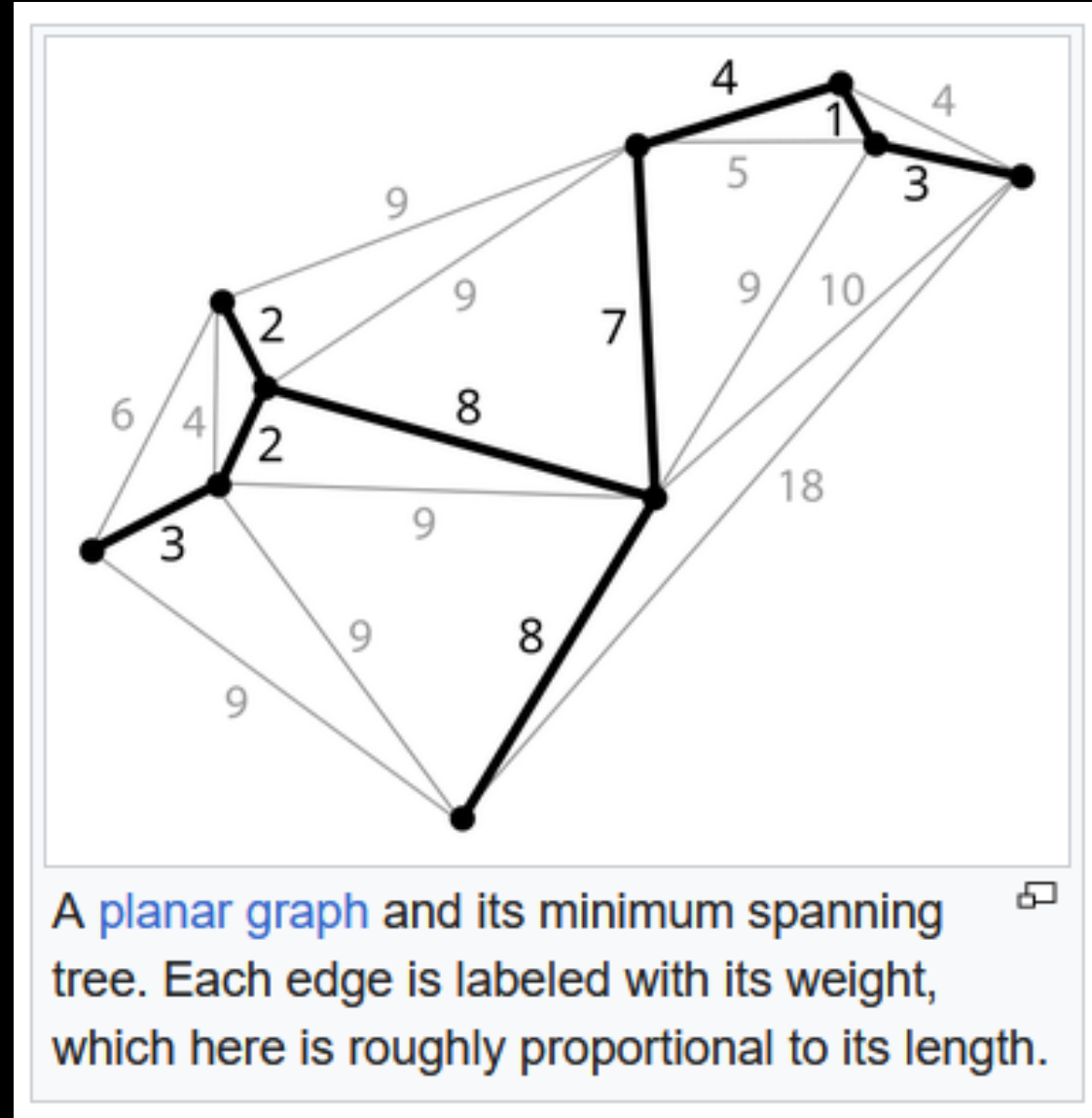
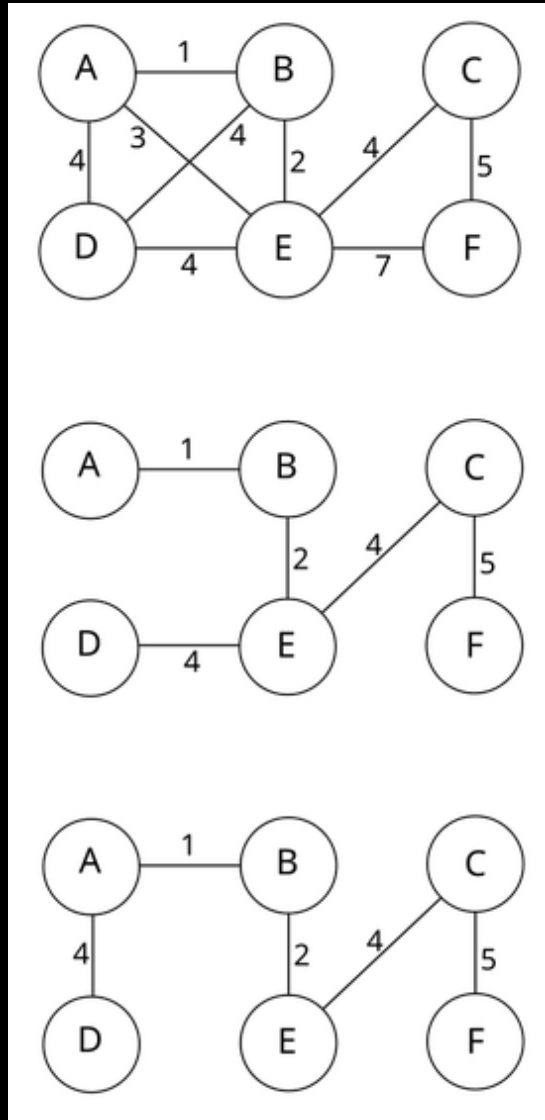
<https://yuminlee2.medium.com/kruskals-algorithm-minimum-spanning-tree-db96e91d0aed>

Minimum Spanning Tree (MST)

1. The minimum spanning tree **has all the properties of a spanning tree**, with an **added constraint of having the minimum possible total edge weight** among all possible spanning trees of that graph.
2. Just like a regular spanning tree, there can be **multiple possible MSTs for a graph** (different trees that share the **same minimum total weight**).



Minimum Spanning Tree (MST)



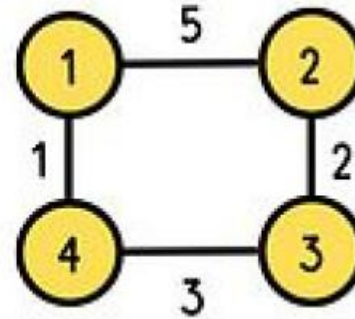
Applications of MST

- i. Designing computer networks (minimum cable cost)
 - i. (MST) connects all nodes in a network with the **minimum total edge cost and no cycles**.
- ii. Telephone and fiber-optic network design
- iii. Electrical power distribution networks
- iv. Water supply pipeline networks
- v. Gas pipeline networks
- vi. Highway or Road network planning
- vii. Railway network design
- viii. Spanning Tree Protocol (STP) for loop-free Ethernet networks
- ix. Network routing and communication infrastructure
- x. Hierarchical clustering in machine learning
- xi. Image segmentation in computer vision
- xii. Approximation algorithms for the Traveling Salesman Problem (TSP)
- xiii. Circuit and PCB (Printed Circuit Board) design
- xiv. Sensor network design
- xv. Distribution and logistics network planning

MST

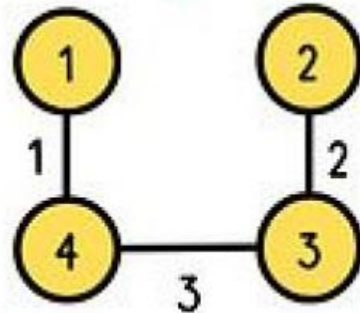
Minimum Spanning Tree(MST)

undirected graph

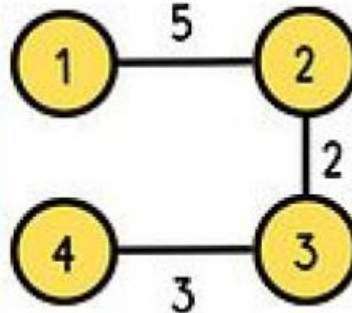


spanning trees

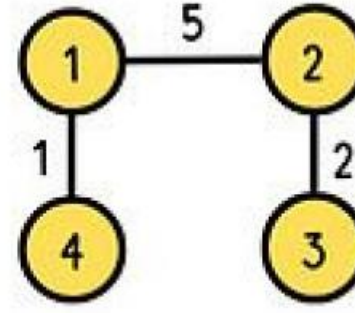
minimum spanning tree



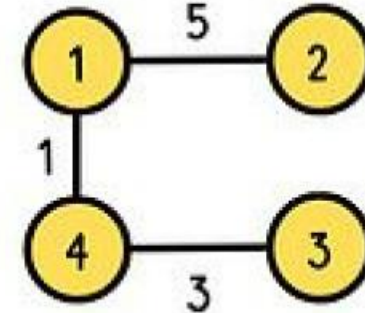
the minimum sum of
all edge weights $\rightarrow 1 + 3 + 2 = 6$



$$3 + 2 + 5 = 10$$



$$1 + 2 + 5 = 8$$



$$1 + 3 + 5 = 9$$

Algorithms to Find the Minimum Spanning Tree

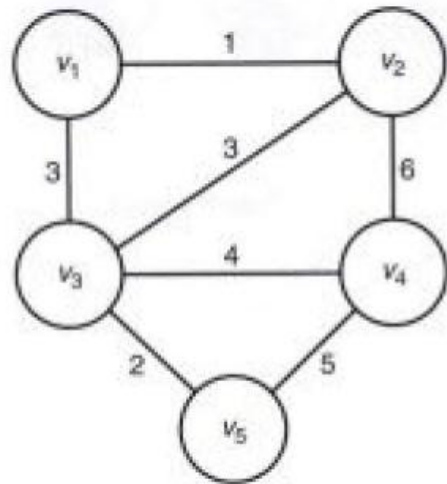
There are several algorithms to find the minimum spanning tree from a given graph. One of the most common is **Kruskal's Minimum Spanning Tree Algorithm** for finding the minimum spanning tree from a connected, undirected graph. It is a **greedy algorithm**.

Algorithm workflow:

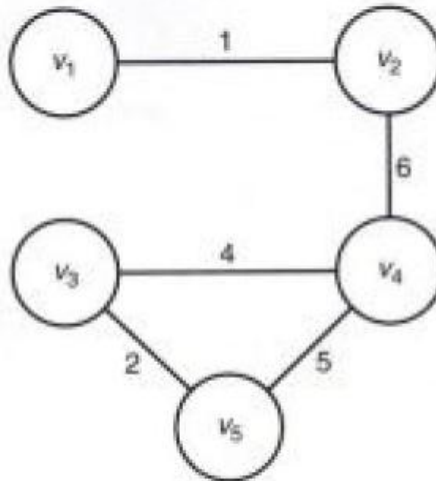
1. **Sort** all the edges of the graph in increasing **order of their weight**.
2. Pick the **smallest edge**. Check **if it forms a cycle** with the **spanning tree formed so far**. If it **does NOT, include** this edge; if it **does, discard** it.
3. **Repeat step 2** until there are **$(V - 1)$ edges** in the spanning tree, where V is the number of vertices in the graph.

Example: A weighted graph and three subgraphs

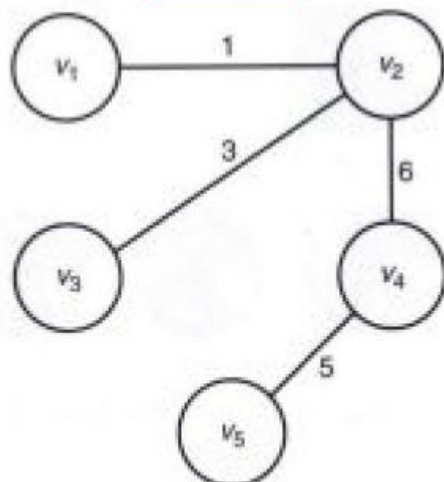
(a) A connected, weighted, undirected graph G .



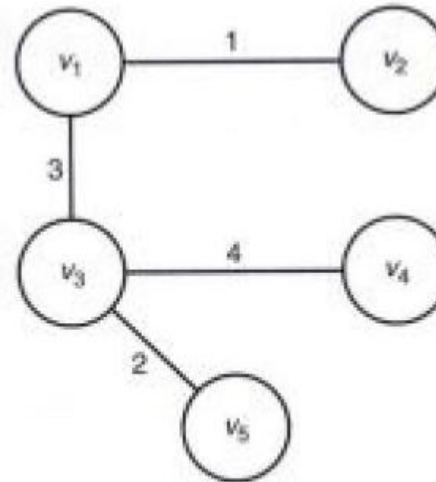
(b) If (v_4, v_5) were removed from this subgraph, the graph would remain connected.



(c) A spanning tree for G .

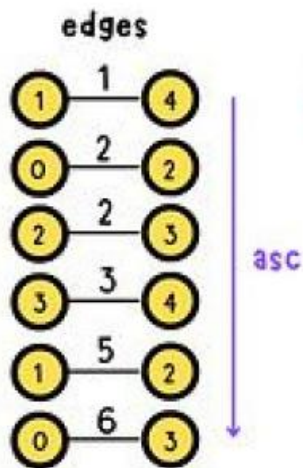
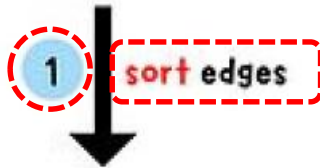
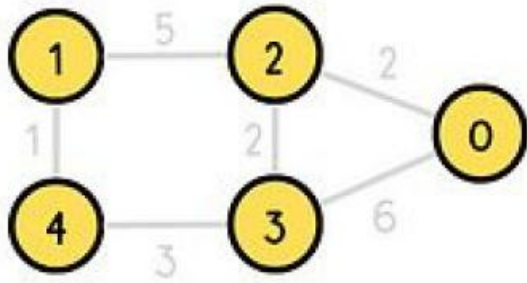


(d) A minimum spanning tree for G .



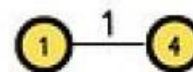
Kruskal's Algorithm

find the **minimum spanning tree(MST)**



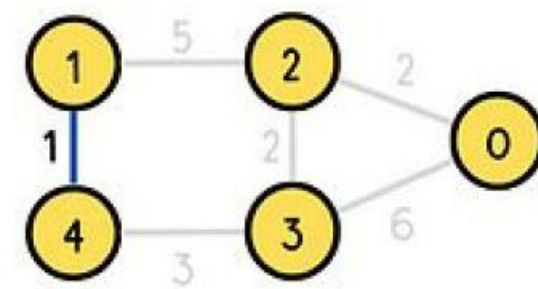
pick the least weight edge (greedy)

2



union find

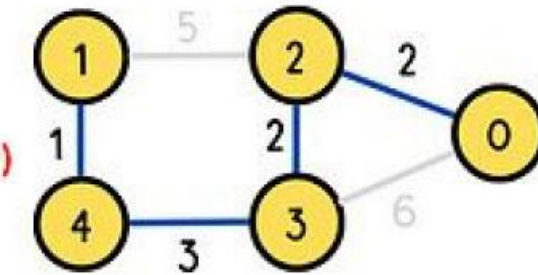
3



two nodes **not** in the same tree
→ **unify**

4 **repeat** 2 3

MST
(minimum total weight)



algorithm is a union find algorithm because it implements find operation to check if nodes are in the same tree/set and **perform union operation** to merge two disjoint trees/sets

Algorithms to Find Minimum Spanning Tree ...

Prim's Minimum Spanning Tree Algorithm

This is also a greedy algorithm. It has the following workflow:

1. Start with **any single vertex**, forming the initial (trivial) MST.
2. Look at **all edges connecting the current MST** to vertices outside it, and **pick** the one with the **minimum weight**.
3. **Add** that **edge** and its vertex to the MST.
4. Repeat steps 2 to 3 **until all vertices are included in the MST**

Algorithms to Find Minimum Spanning Tree ...

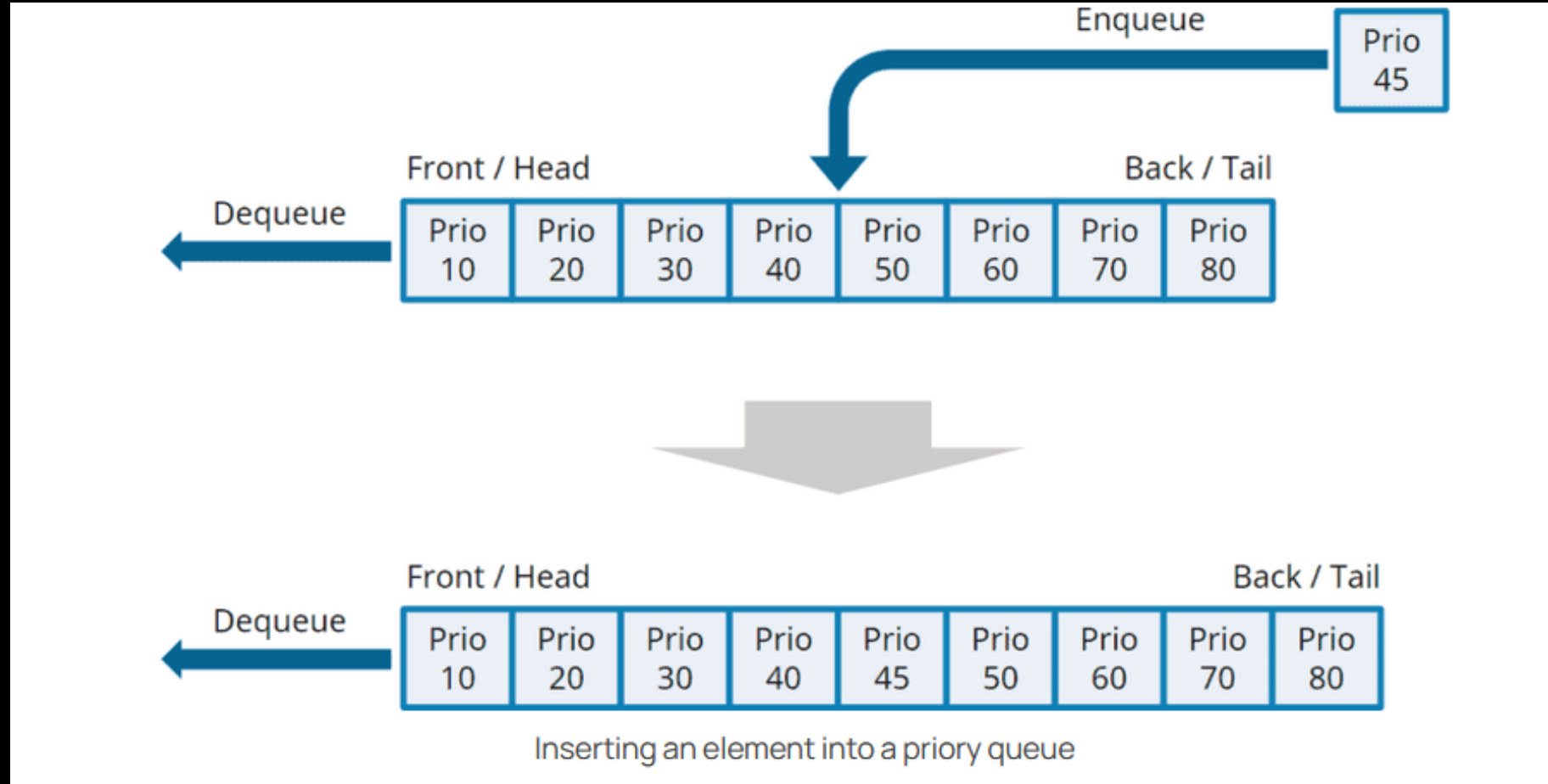
To efficiently select the minimum weight edge at each iteration, this algorithm uses **a priority queue to store the vertices sorted by their minimum edge weight** at that point.

It also **simultaneously keeps track of the growing MST using an array or another suitable data structure**, depending on the graph representation.

Applications: This algorithm can be used in various scenarios such as **image segmentation based on color, texture, or other features, and routing**, (for example, finding the shortest path between two points for a delivery truck to follow.)

Time complexity: Both Kruskal's and Prim's algorithms have a time complexity of **$O(E \log E)$** , where E is the **number of edges** in the graph.

Priority Queue



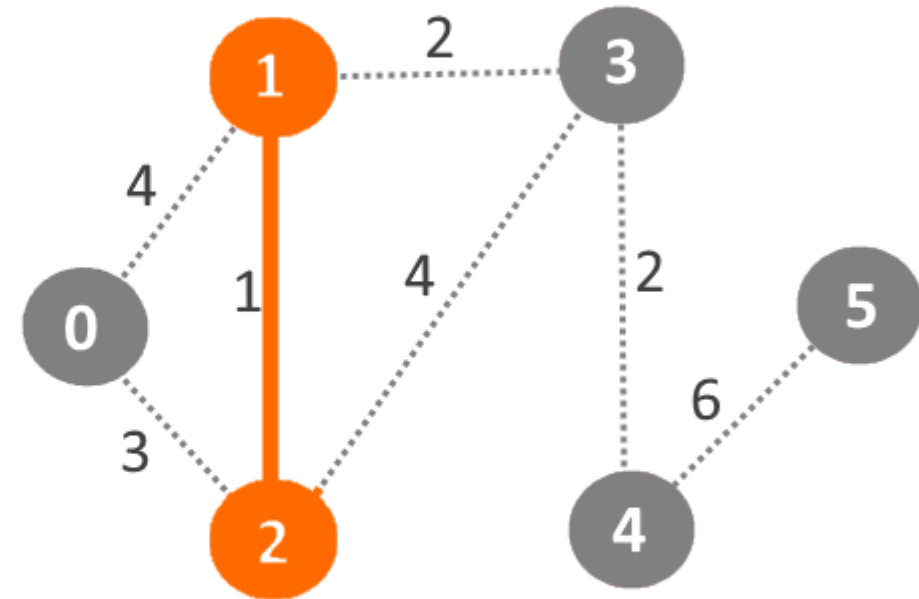
<https://www.happycoders.eu/algorithms/priorityqueue-java/>

Kruskal's Algo

Steps

1. Sort the edges in ascending order of weights.
2. Pick the edge with the least weight. Check if including this edge in spanning tree will form a cycle is Yes then ignore it if No then add it to spanning tree.
3. Repeat the step 2 untill spanning tree has $V-1$ edges (V – vertices in Graph).

Edges	1-2	1-3	3-4	0-2	0-1	2-3	4-5
Weights	1	2	2	3	4	4	6



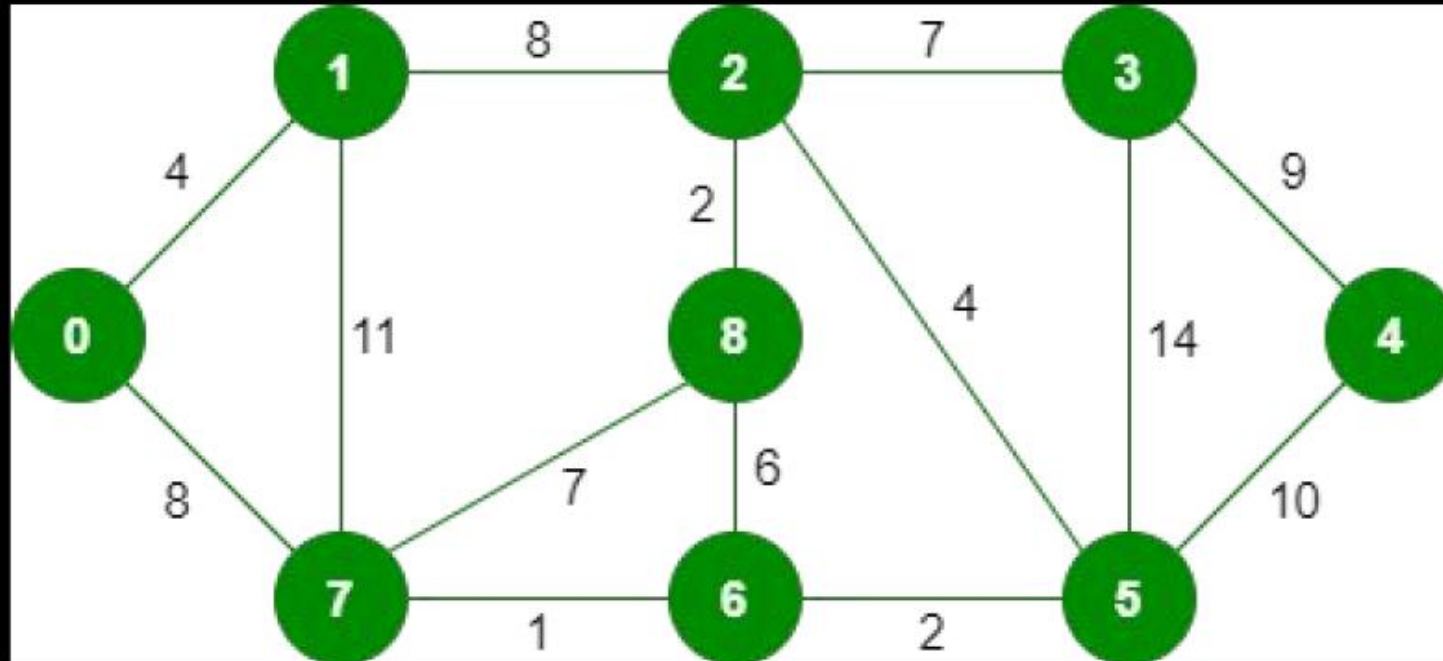
Edge 0-1:

No cycle formed by including edge 0-1 so include it

MST Edges : 1

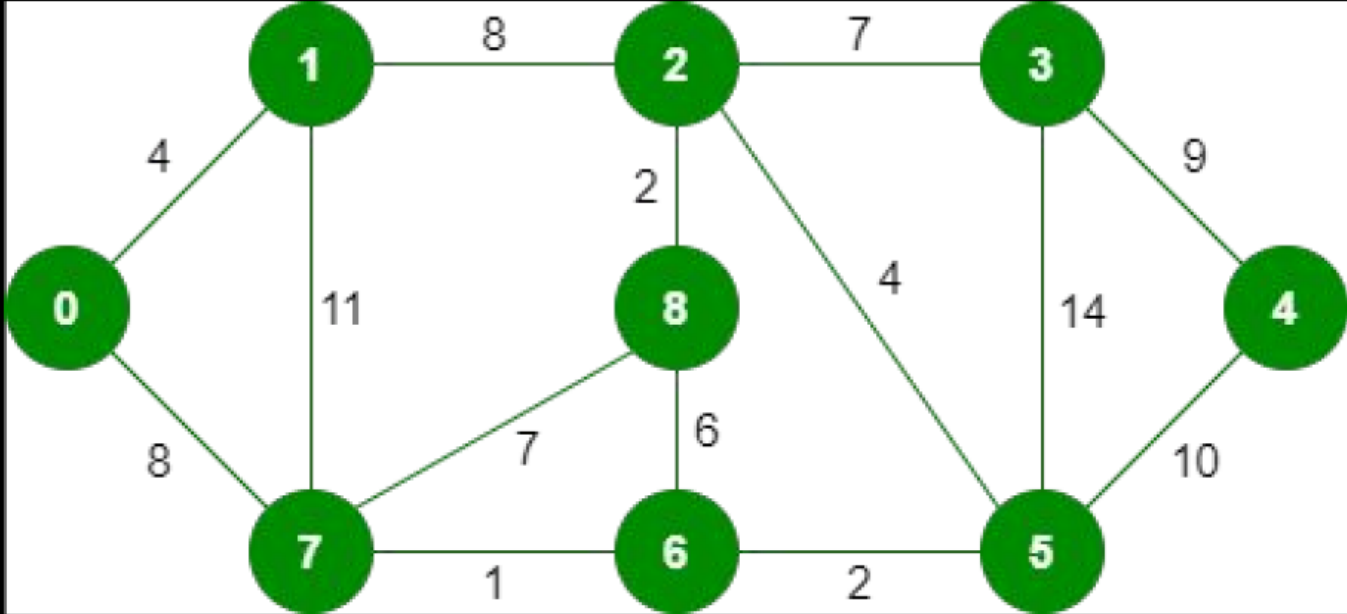
Kruskal's algorithm

- **Kruskal's algorithm** to find the minimum cost spanning tree uses the **greedy** approach.
- The **Greedy Choice is to pick the smallest weight edge** that **does NOT cause a cycle** in the MST constructed so far.



Weight	Source Vertex	Destination Vertex
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

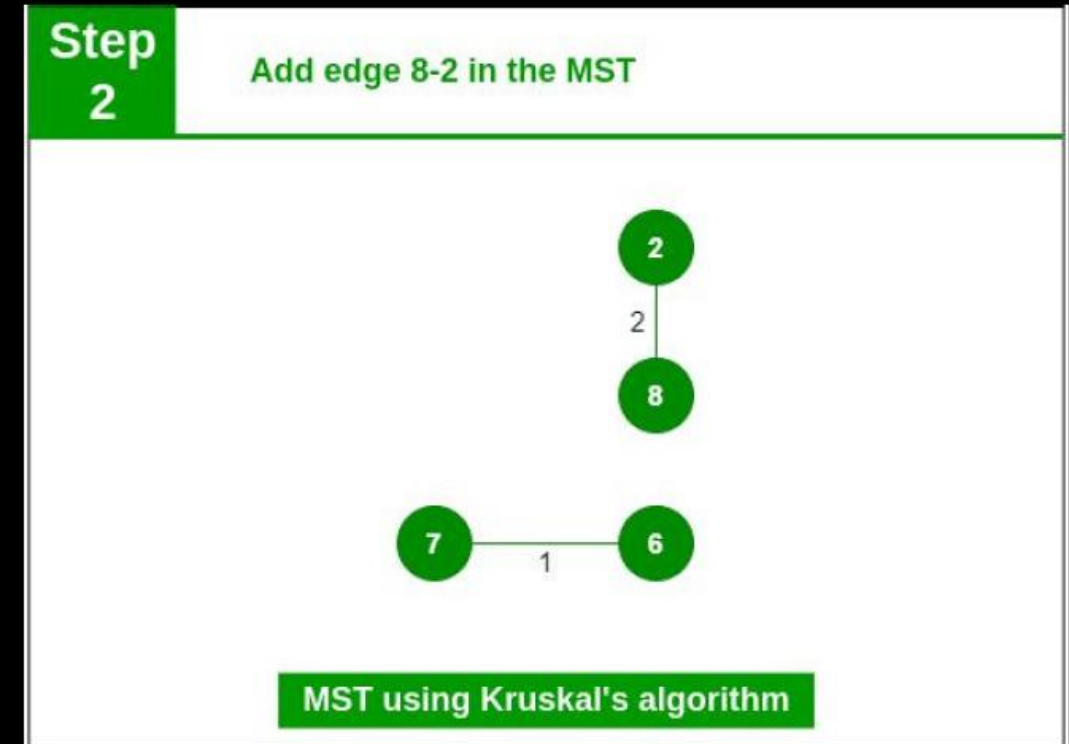
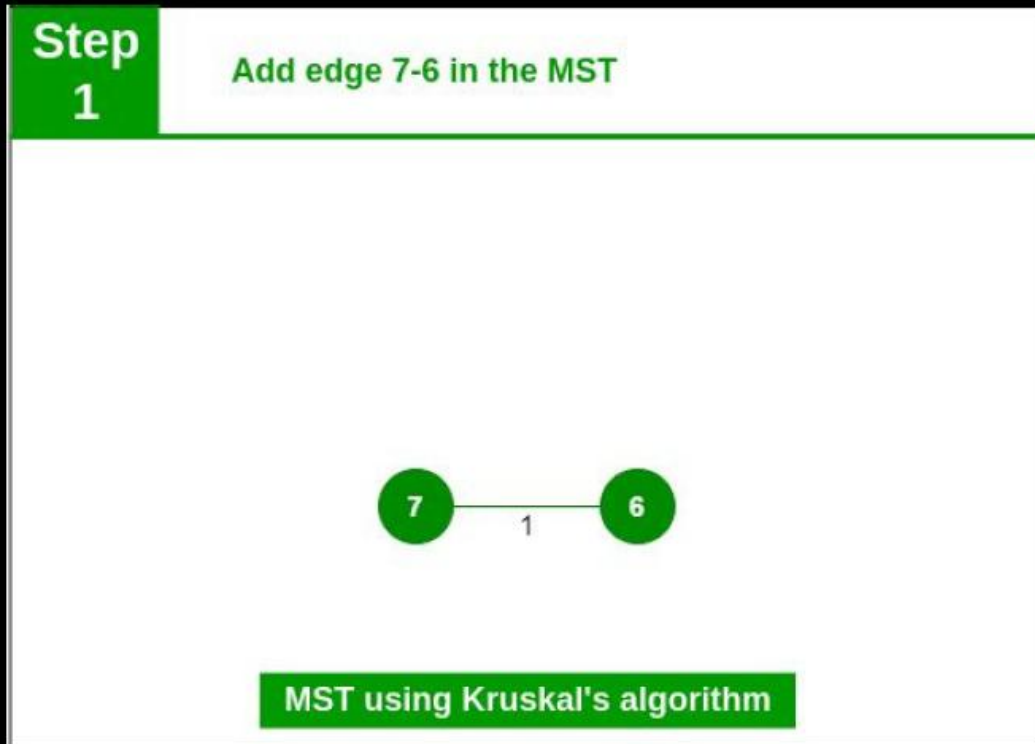
Kruskal's algorithm example



Kruskal's algorithm example ctd..

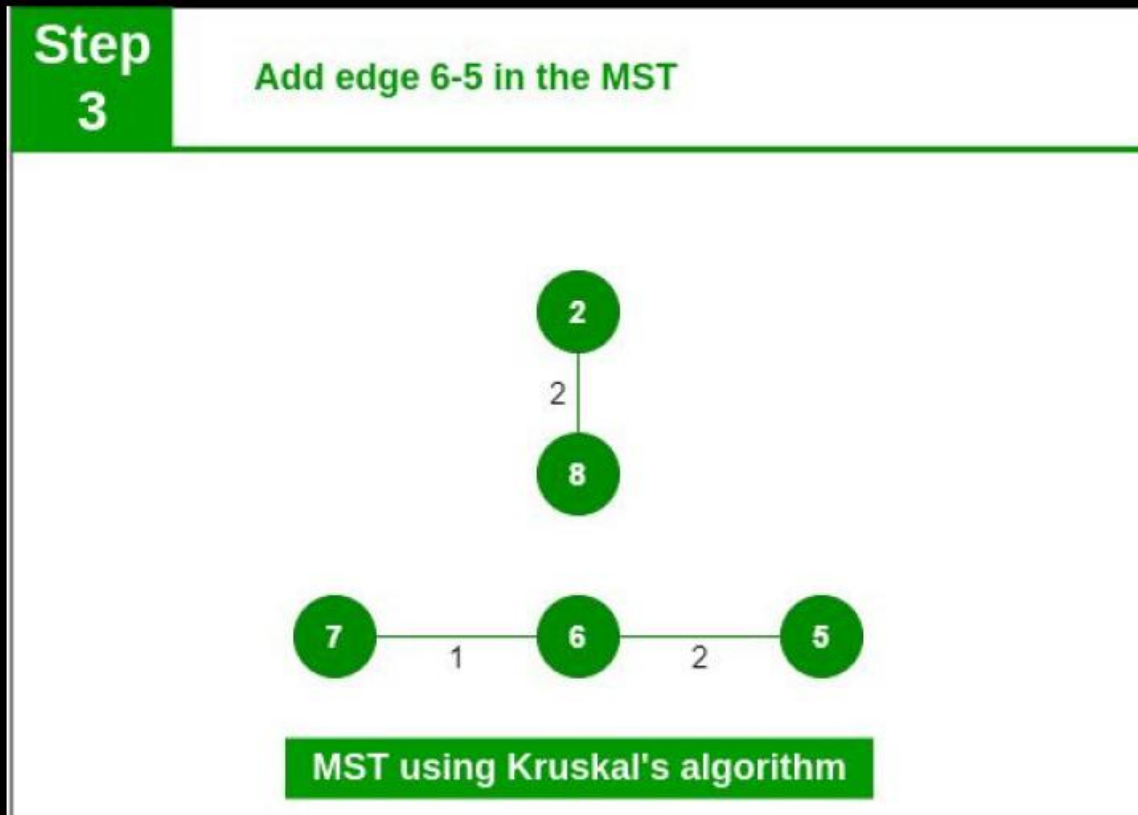
- Now pick all edges one by one from the sorted list of edges
- **Step 1:** Pick edge 7-6. No cycle is formed, include it.

- **Step 2:** Pick edge 8-2. No cycle is formed, include it.

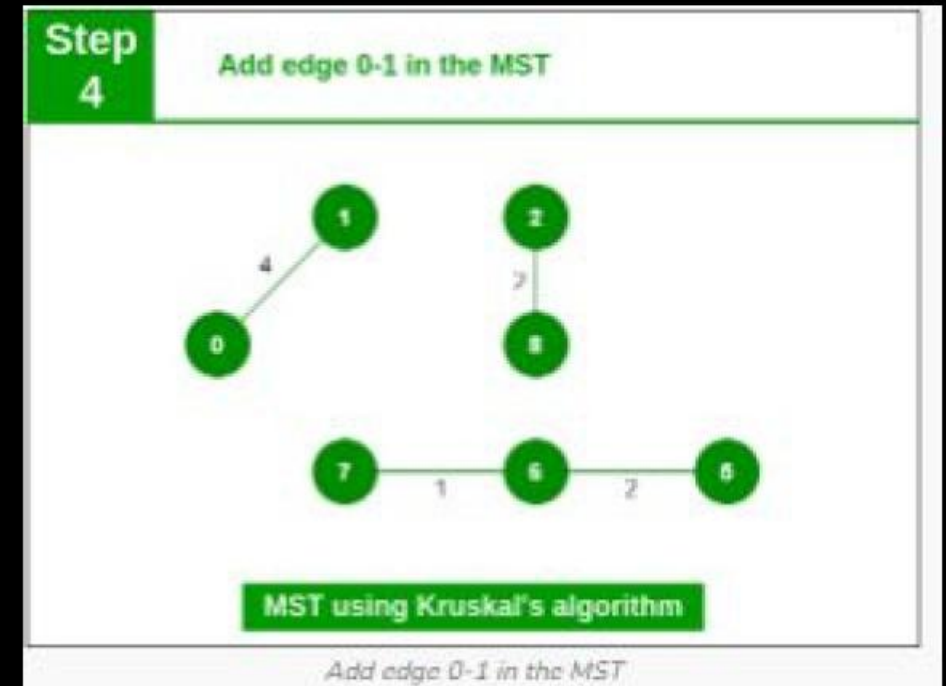


Kruskal's algorithm example ctd..

- Step 3: Pick edge 6-5. No cycle is formed, include it.

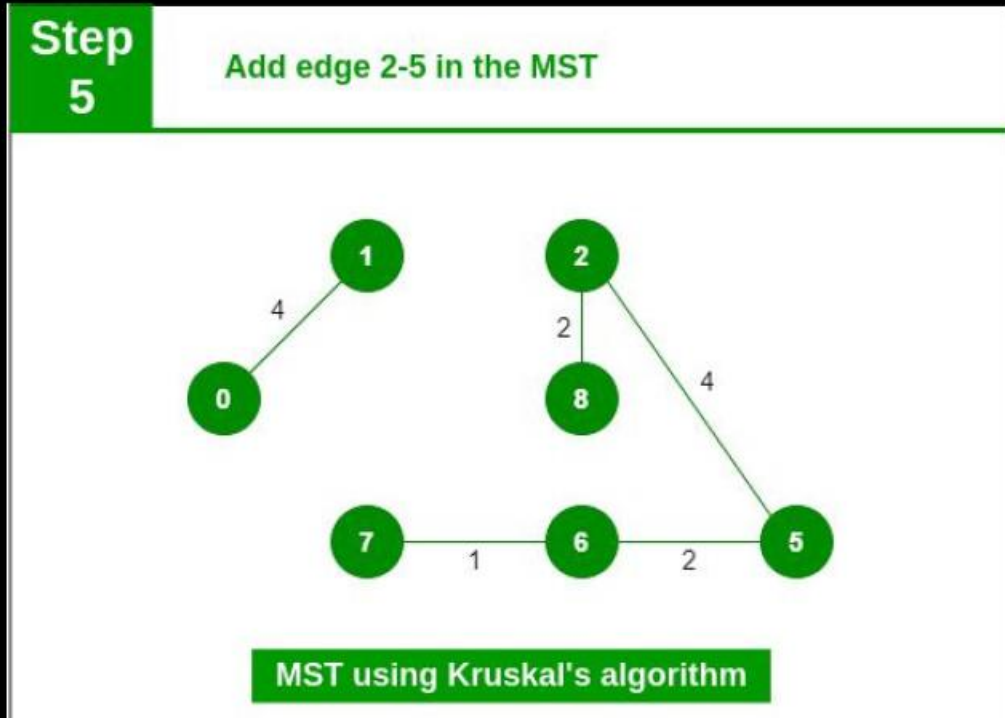


- Step 4: Pick edge 0-1. No cycle is formed, include it.

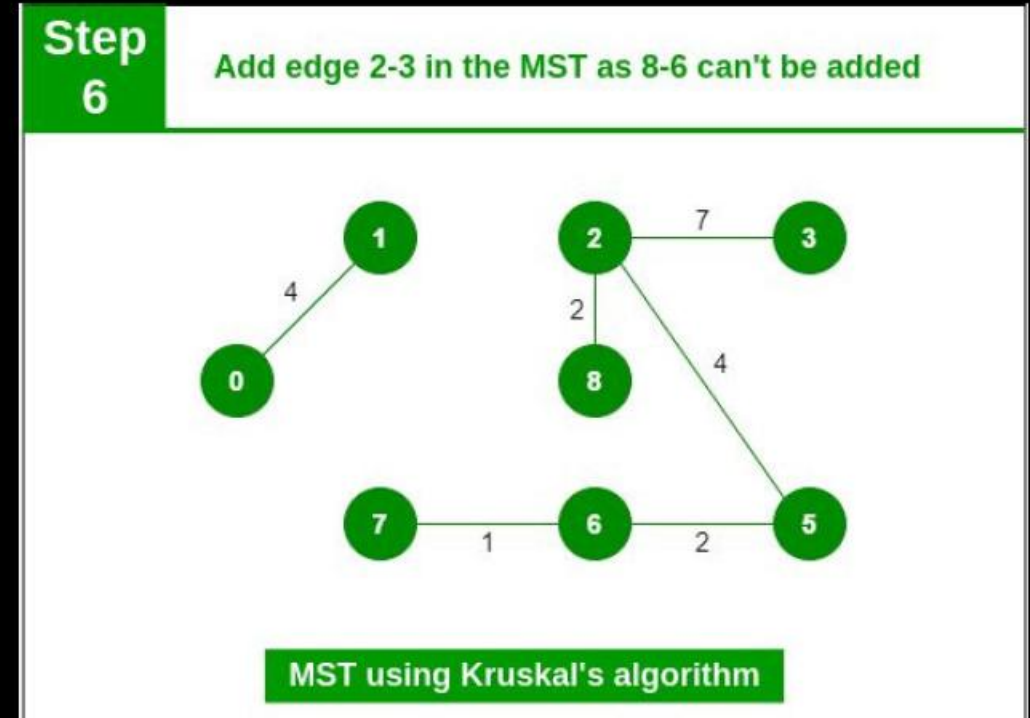


Kruskal's algorithm example ctd..

- Step 5: Pick edge 2-5. No cycle is formed, include it.

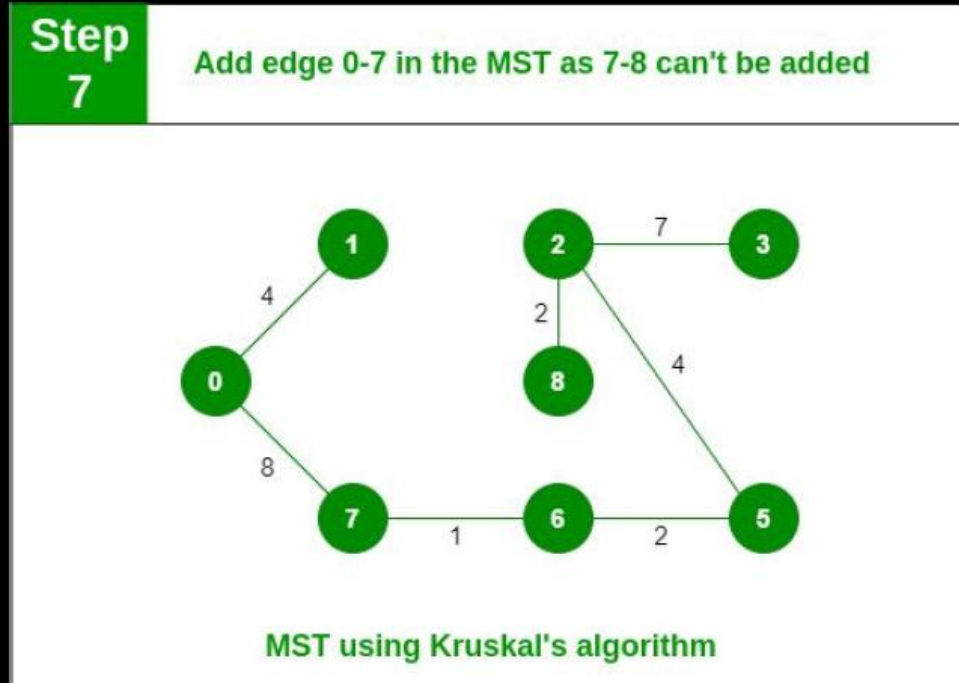


- Step 5: Pick edge 2-5. No cycle is formed, include it.

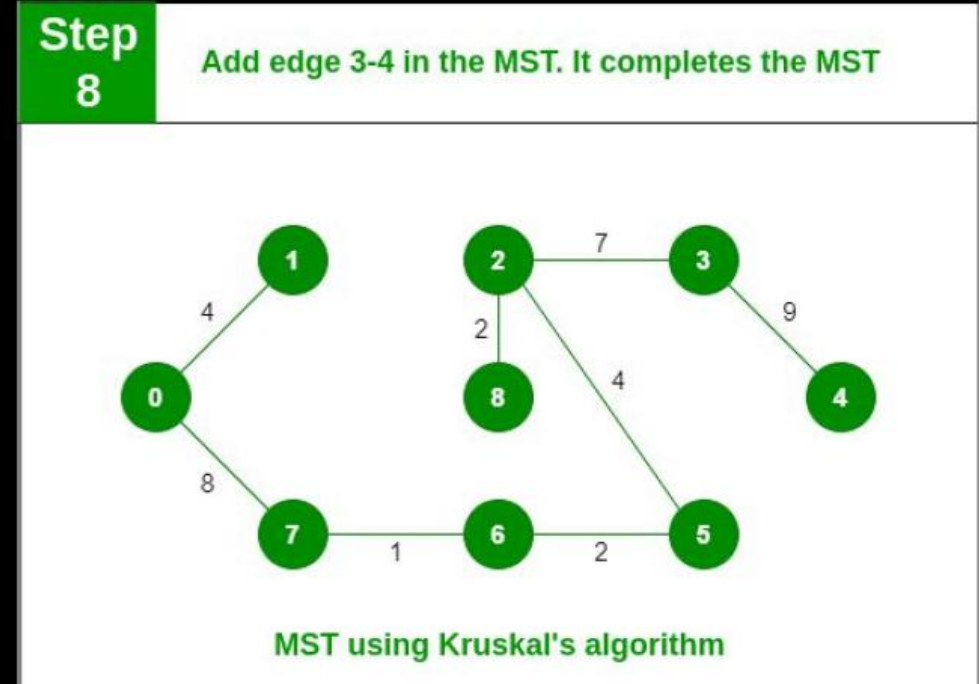


Kruskal's algorithm example ctd..

- **Step 7:** Pick edge 7-8. Since including this edge results in the cycle, discard it. Pick edge 0-7. No cycle is formed, include it.



- **Step 8:** Pick edge 1-2. Since including this edge results in the cycle, discard it. Pick edge 3-4. No cycle is formed, include it.



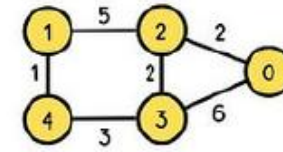
Note: Since the number of edges included in the MST equals to $(V - 1)$, so the algorithm stops here
** the number of edges in the MST is always $V - 1$, where V is the number of vertices in a connected undirected graph.

Further Example

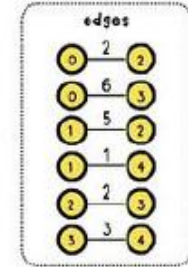
- Ref : Kruskal's Algorithm: Minimum Spanning Tree

<https://yuminlee2.medium.com/kruskals-algorithm-minimum-spanning-tree-db96e91d0aed>

- **step1:** Sort edges by ascending edge weight
- **step2:** Initialize **parent** and **size** arrays with the length of the total number of nodes.

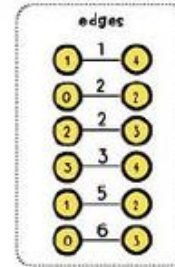


numOfNodes = 5



step 1:

Sort edges by ascending edge weight

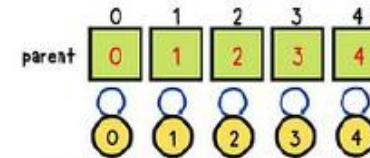


step 2:

Initialize **parent** and **size** arrays with the length of the total number of nodes.

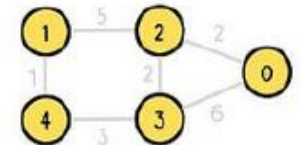
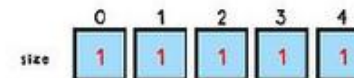
step 2a:

Originally every node is a root node to itself



step 2b:

Originally every tree/set contains a single node



- **step3:** Initialize a variable **edgeCount** to 0
- **step4:** Pick the smallest edge

Kruskal's Algorithm Process

step 3:

Initialize a variable **edgeCount** to 0

$\text{edgeCount} = 0$

step 4:

Pick the smallest edge

step 4.a:

find root parent and check if two nodes are in the same tree

$\text{root1} == \text{root2}?$

Yes

in the same tree $\rightarrow$ return false

No

step 4.b:

merge the smaller tree to the larger tree

$\text{parent}[\text{root1}] = \text{root2}$ or $\text{parent}[\text{root2}] = \text{root1}$

step 4.c:

increment the size of the larger tree by 1

$\text{size}[\text{root2}] += 1$ or $\text{size}[\text{root1}] += 1$

return true

step 4.d:

increment **edgeCount** by 1 if return true

$\text{edgeCount} += 1$

- **step5:** repeat **step4** until it includes (the number of nodes -1) edges in the MST

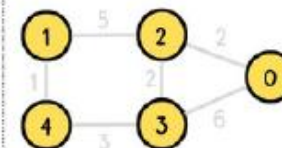
step 4:

Pick the smallest edge



	0	1	2	3	4
parent	0	1	2	3	4
size	1	1	1	1	1

edges	
1	4
0	2
2	3
3	4
1	5
0	6



step 4.a:

find root parent and check if two nodes are in the same tree

$\text{root1} = \text{find}(1) = 1$
 $\text{root2} = \text{find}(4) = 4$
 $\text{root1} \neq \text{root2}$

step 4.b:

merge the smaller tree to the larger tree

$\text{size}[\text{root1}] = \text{size}[1] = 1$
 $\text{size}[\text{root2}] = \text{size}[4] = 1$
 same size $\rightarrow$ just choose one of sets to be the root
 $\text{parent}[\text{root2}] = \text{root1} \rightarrow \text{parent}[4] = 1$

	0	1	2	3	4
parent	0	1	2	3	1

step 4.c:

increment the size of the larger tree by 1

	0	1	2	3	4
size	1	2	1	1	1

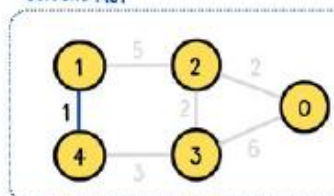
return true

step 4.d:

increment edgeCount by 1 if return true

$\text{edgeCount} += 1 \rightarrow \text{edgeCount} = 1$

current MST



Kruskal's Algorithm Process

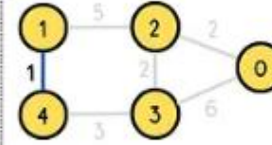
step 4:

Pick the smallest edge



	0	1	2	3	4
parent	0	1	2	3	1
size	1	2	1	1	1

edges			
1	1	4	
0	2	2	
2	2	3	
3	3	4	
1	5	2	
0	6	3	



step 4.a:

find root parent and check if two nodes are in the same tree

$\text{root1} = \text{find}(0) = 0$
 $\text{root2} = \text{find}(2) = 2$
 $\text{root1} \neq \text{root2}$

step 4.b:

merge the smaller tree to the larger tree

$\text{size}[\text{root1}] = \text{size}[0] = 1$
 $\text{size}[\text{root2}] = \text{size}[2] = 1$
 same size $\rightarrow$ just choose one of sets to be the root
 $\text{parent}[\text{root2}] = \text{root1} \rightarrow \text{parent}[2] = 0$

	0	1	2	3	4
parent	0	1	0	3	1

step 4.c:

increment the size of the larger tree by 1

$\text{size}[\text{root1}] += 1 \rightarrow \text{size}[0] += 1$

	0	1	2	3	4
size	2	2	1	1	1

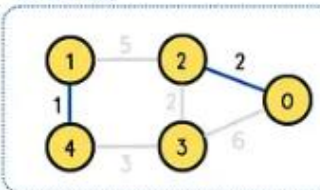
return true

step 4.d:

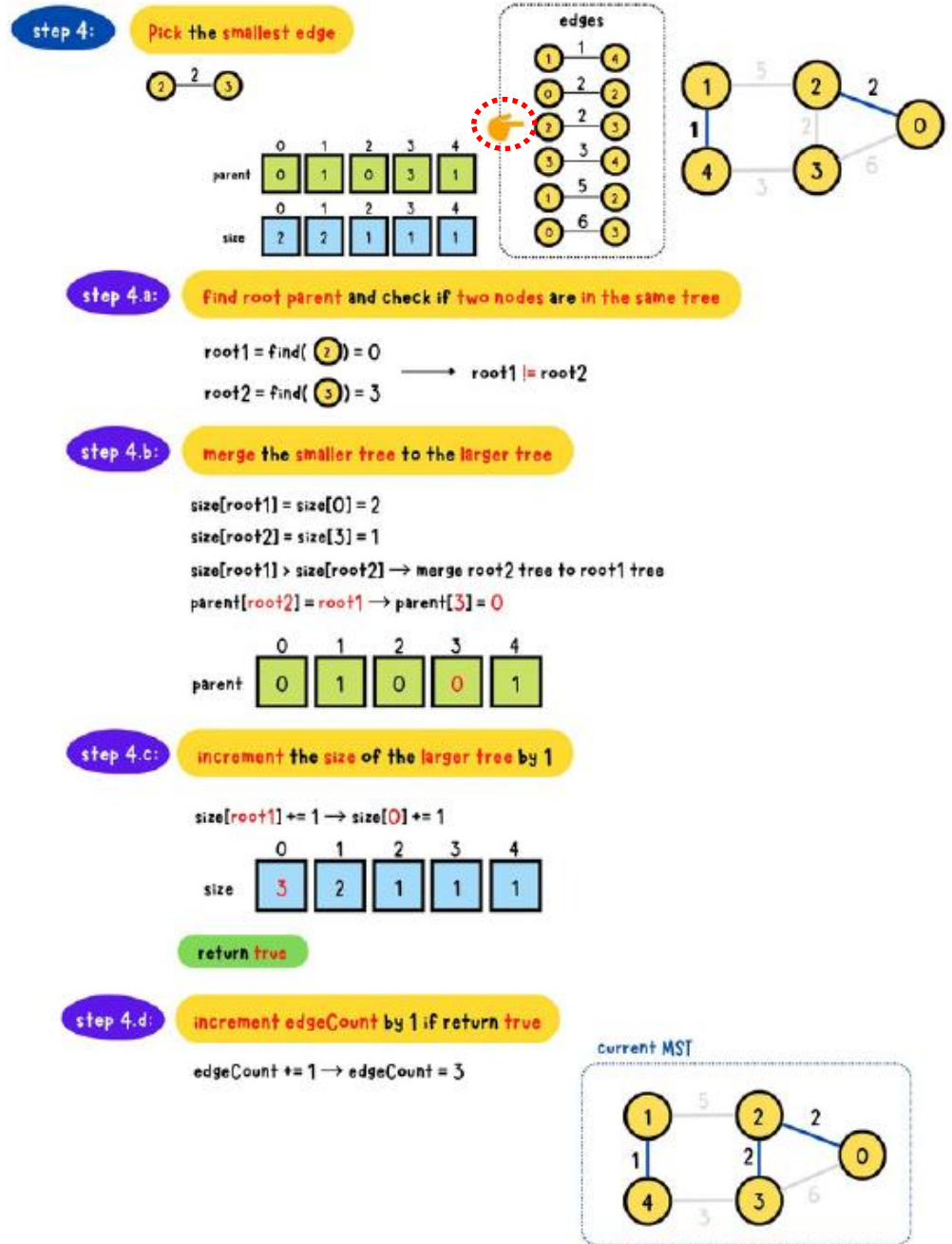
increment edgeCount by 1 if return true

$\text{edgeCount} += 1 \rightarrow \text{edgeCount} = 2$

current MST



- **step5:** repeat **step4** until it includes (the number of nodes -1) edges in the MST



Kruskal's Algorithm

```
Kruskal(G):  
    MST = empty set  
    Sort all edges in increasing order of weight  
    Make a separate set for each vertex  
    For each edge (u, v) in sorted order:  
        If Find(u) ≠ Find(v):  
            Add edge (u, v) to MST  
            Union(u, v)  
        If MST contains  $V - 1$  edges:  
            Stop  
    Return MST
```

Kruskal's Algorithm Complexity

- Time Complexity: $O(E \log E)$ (or equivalently $O(E \log V)$)
 - Sorting all E edges: $O(E \log E)$ (dominant step)
 - Union-Find operations: $O(E \alpha(V)) \approx O(E)$
- Space Complexity: $O(V)$
 - Parent array: $O(V)$
 - Rank array: $O(V)$

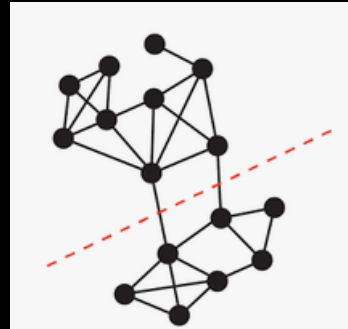
Where:

- V = Number of vertices
- E = Number of edges

History and Key Concept of Prim's Algorithm

- Prim's algorithm was originally invented in 1930 by the Czech mathematician Vojtěch Jarník.
- The algorithm was later rediscovered by Robert C. Prim in 1957, and again independently by Edsger W. Dijkstra in 1959. Because of this shared history, the algorithm is sometimes also called *Jarník's algorithm* or the *Prim-Jarník algorithm*.

Cut: In graph theory, a **group of edges that connects two sets of vertices** in a graph is called a cut.



At every step of Prim's algorithm:

1. **Find a cut**, one set being the vertices **already in the MST**, and the other being the **remaining vertices**.
2. **Pick the minimum weight edge crossing that cut**.
3. **Include the new vertex** (connected by that edge) into the MST set (the set of vertices already included in the tree).

Prim's Algorithm for Minimum Spanning Tree (MST)

- Prim's algorithm is a **Greedy algorithm** used to find a Minimum Spanning Tree(**MST**).
 - Always choose the **minimum-weight edge** connecting the current MST to a new vertex (**greedy choice**).
- It starts from **any one vertex** and grows the MST by adding one vertex at a time.
- The algorithm maintains **two sets of vertices**:
 - **MST Set**: Vertices **already included** in the MST.
 - **Non-MST Set**: Vertices **not yet** included.
- At each step:
 1. Find the **minimum-weight edge that connects a vertex in the MST** set to a vertex **outside the MST**.
 2. **Add** that edge and the new vertex to the **MST**.
 3. Repeat until **all** vertices are included in the MST.

Prim's Algorithm

1. Choose an **arbitrary vertex** as the starting vertex.
2. **Initialize** the **MST** with the **starting** vertex.
3. Find **all edges** connecting a vertex in the MST to a vertex **NOT yet in the MST**.
4. **Select** the edge with the **minimum** weight.
5. **Add** the selected **edge** and its new vertex to the MST (this will never form a cycle, since the new vertex is outside the current MST).
6. Repeat steps 3 to 5 **until all vertices** are included in the MST.
7. Return the completed Minimum Spanning Tree (MST).

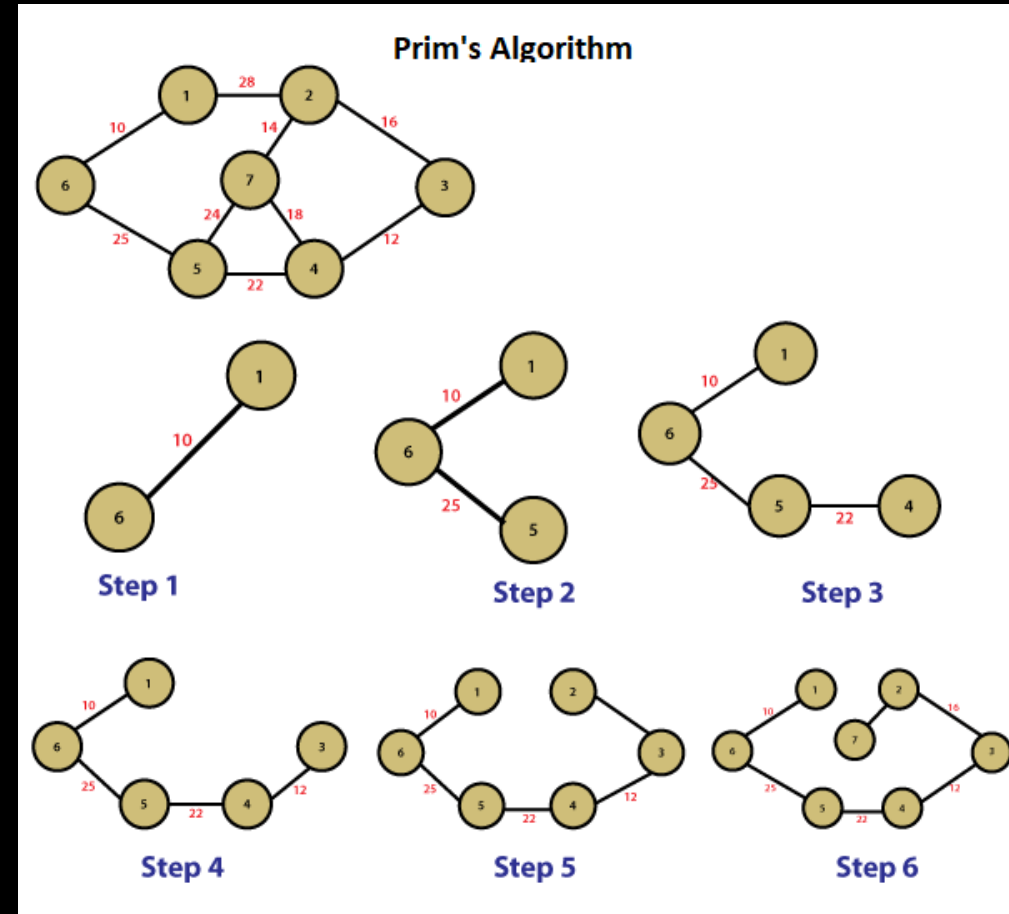
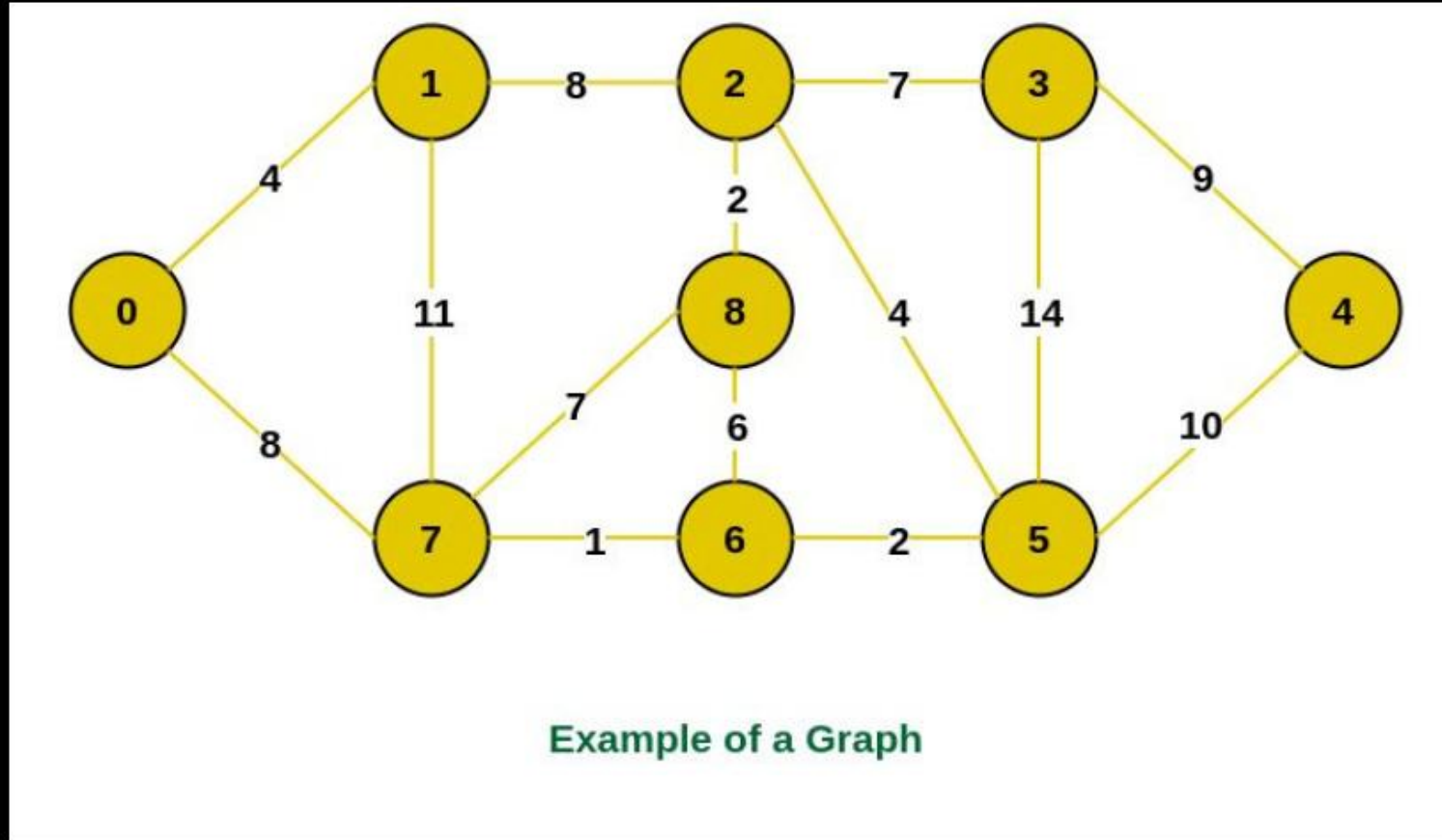


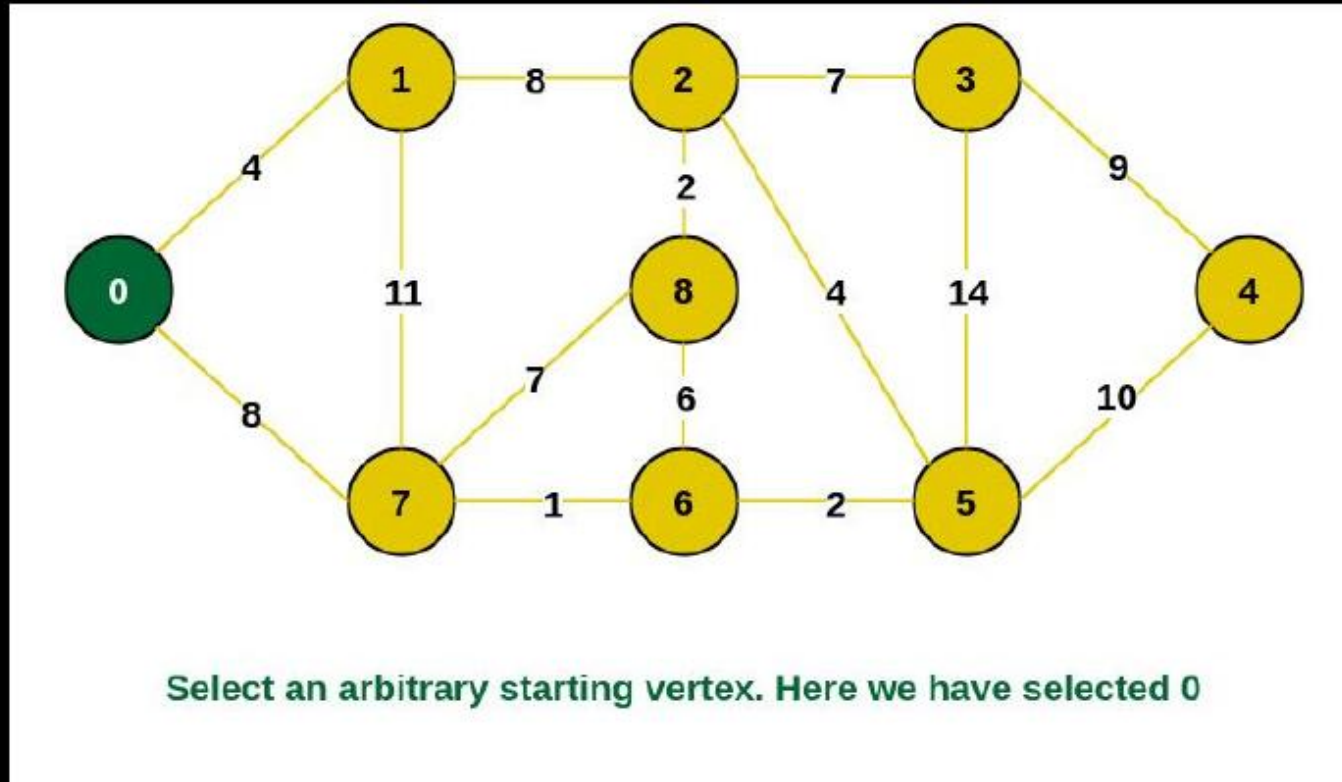
Illustration of Prim's Algorithm:

- Consider the following graph as an example for which we need to find the Minimum Spanning Tree (MST).



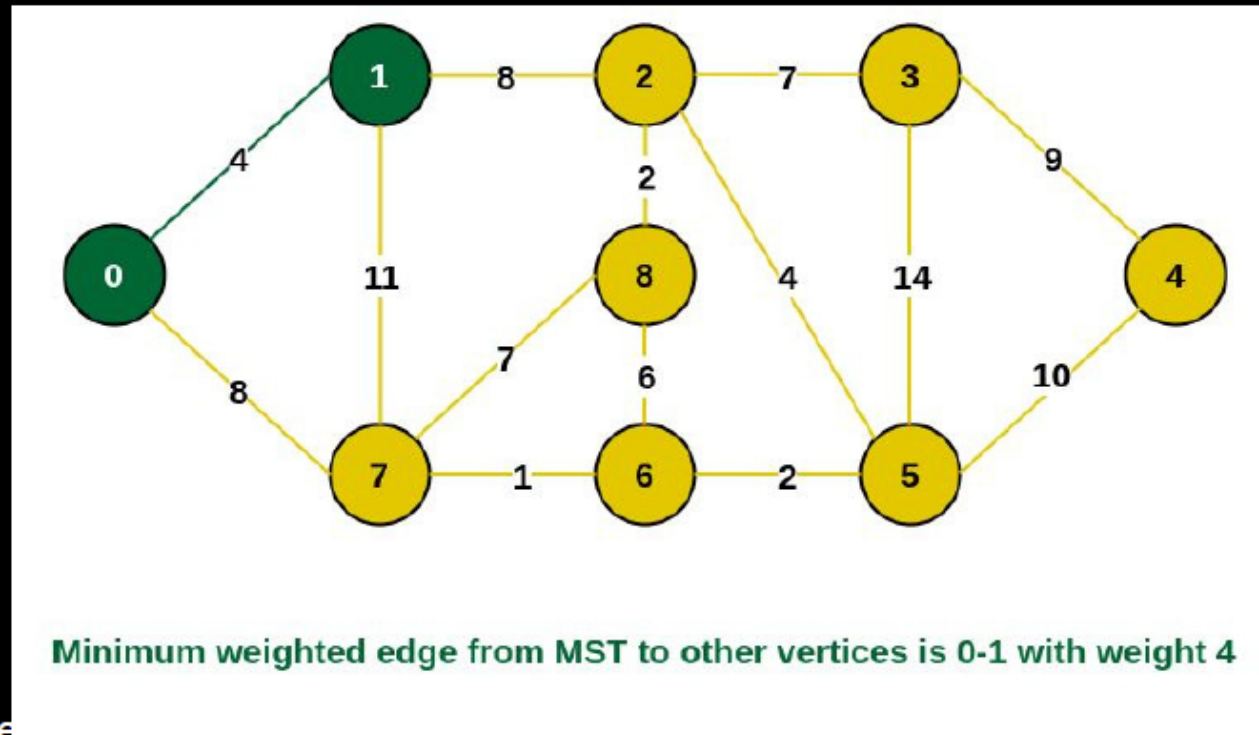
Prim's Algorithm

- Step1: Firstly, we select an arbitrary vertex that acts as the starting vertex of the Minimum Spanning Tree. Here we have selected vertex 0 as the starting vertex.



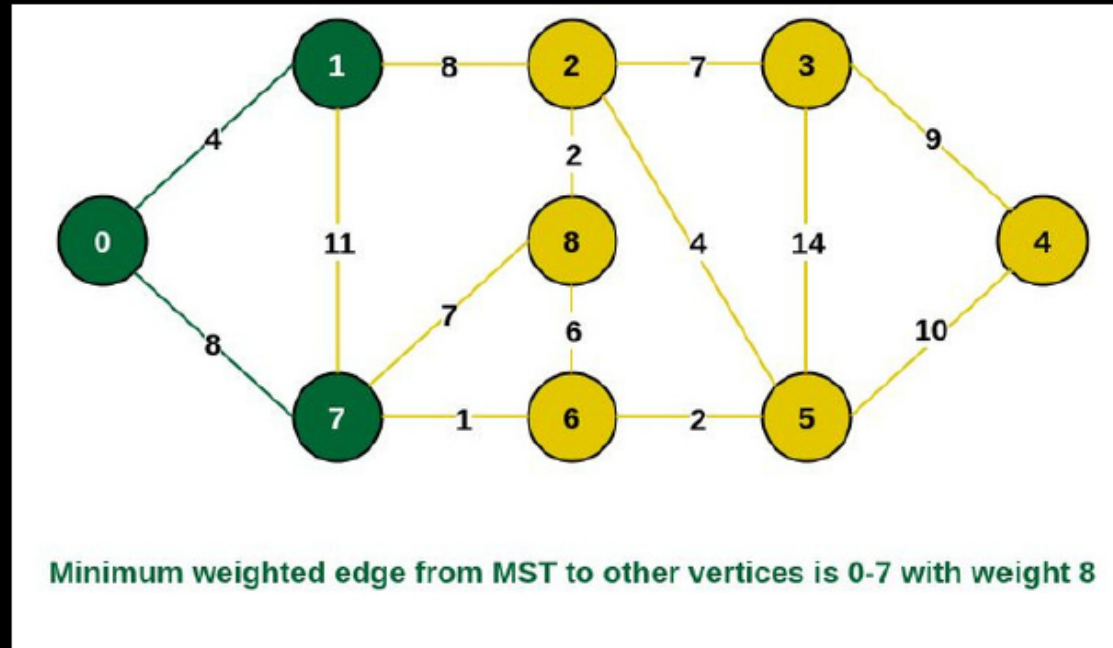
Prim's Algorithm ctd..

- Step 2: All the edges connecting the incomplete MST and other vertices are the edges {0, 1} and {0, 7}. Between these two the edge with minimum weight is {0, 1}. So include the edge and vertex 1 in the MST.



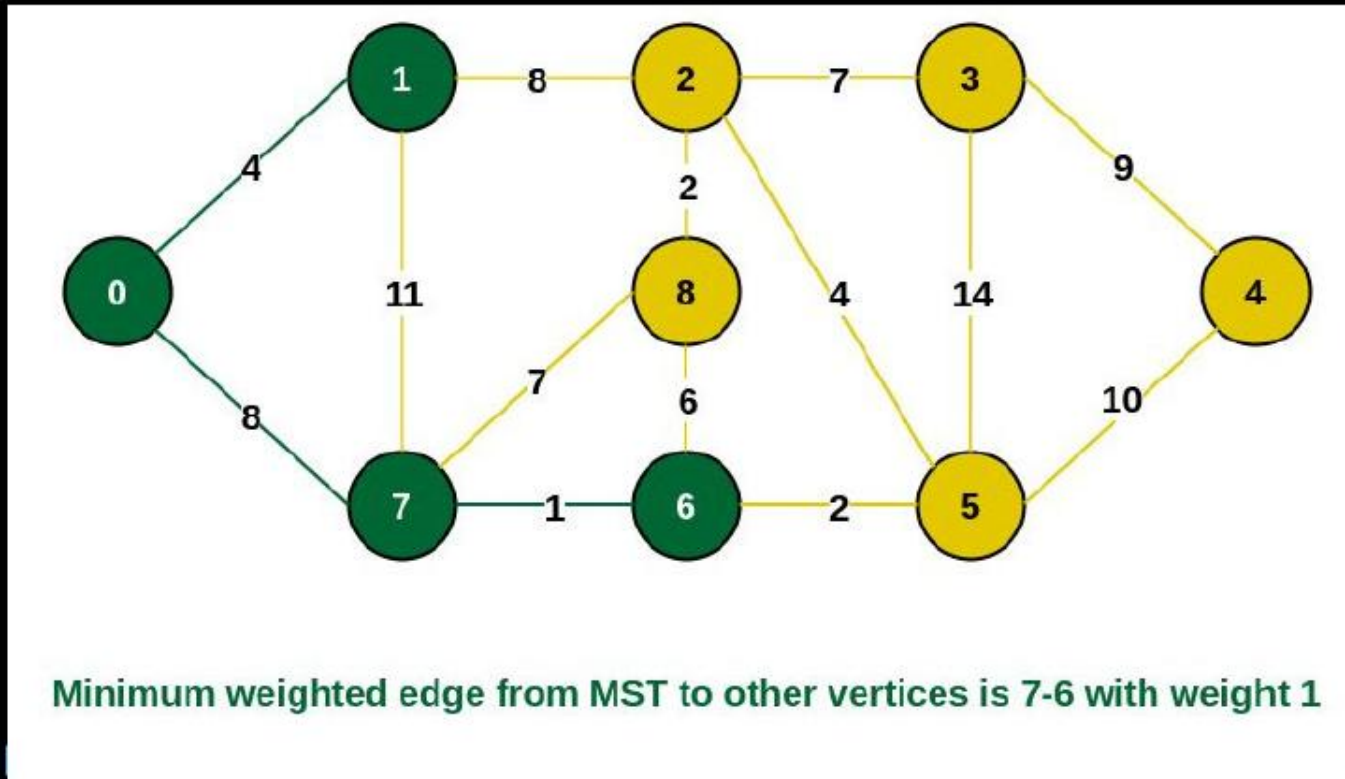
Prim's Algorithm ctd..

- Step 3: The edges connecting the incomplete MST to other vertices are $\{0, 7\}$, $\{1, 7\}$ and $\{1, 2\}$. Among these edges the **minimum weight is 8 which is of the edges $\{0, 7\}$ and $\{1, 2\}$** . Let us here include the edge $\{0, 7\}$ and the vertex 7 in the MST. [We could have also included edge $\{1, 2\}$ and vertex 2 in the MST].



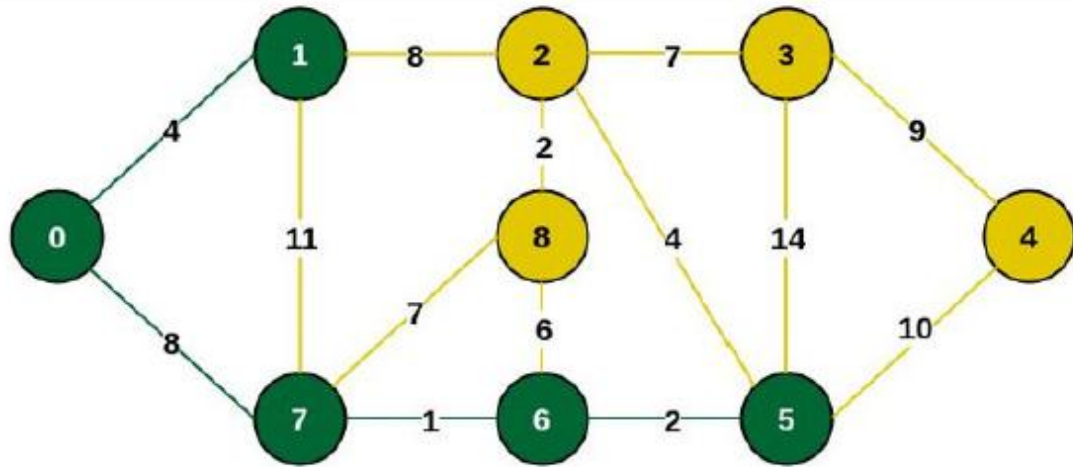
Prim's Algorithm ctd..

- Step 4: The edges that connect the incomplete MST with **the fringe vertices** are **{1, 2}**, **{7, 6}** and **{7, 8}**. Add the **edge {7, 6}** and the **vertex 6** in the MST as it has the **least weight** (i.e., 1).



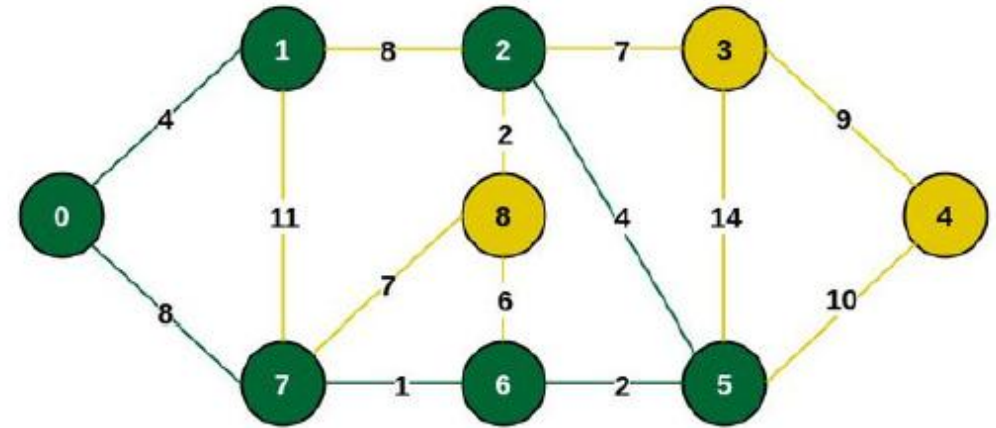
Prim's Algorithm ctd..

- Step 5: The connecting edges now are {7, 8}, {1, 2}, {6, 8} and {6, 5}. Include edge {6, 5} and vertex 5 in the MST as the edge has the minimum weight (i.e., 2) among them.



Minimum weighted edge from MST to other vertices is 6-5 with weight 2

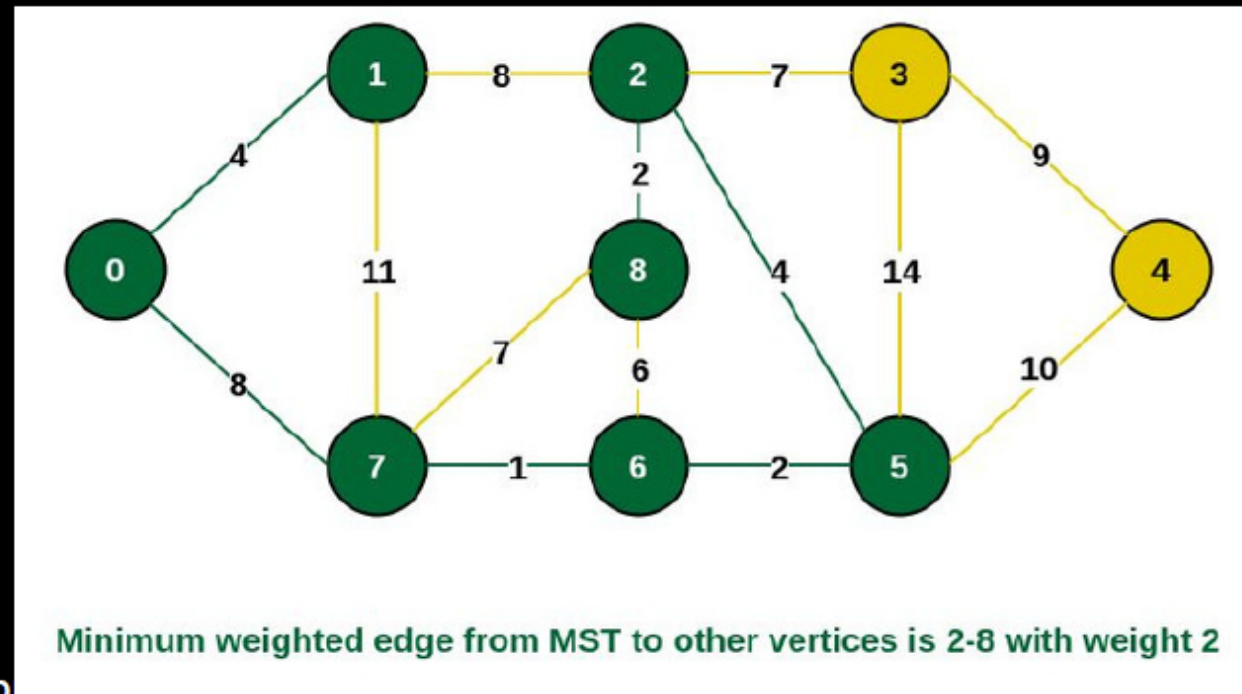
- Step 6: Among the current connecting edges, the edge {5, 2} has the minimum weight. So include that edge and the vertex 2 in the MST.



Minimum weighted edge from MST to other vertices is 5-2 with weight 4

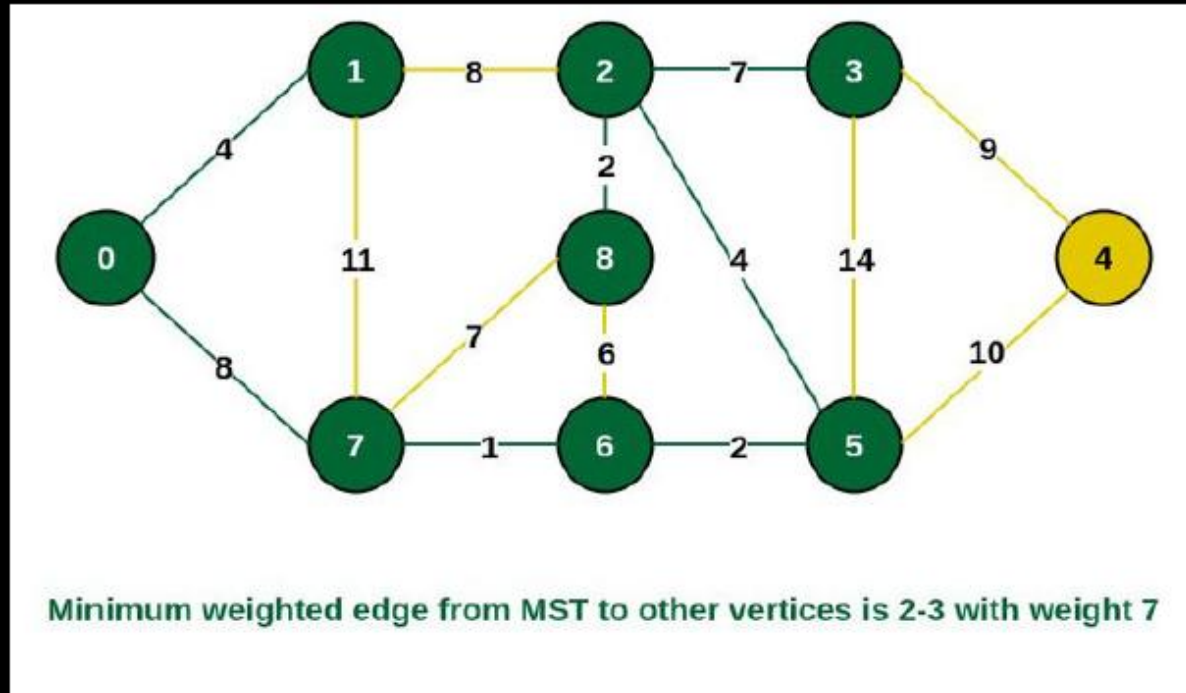
Prim's Algorithm ctd..

- Step 7: The connecting edges between the incomplete MST and the other edges are {2, 8}, {2, 3}, {5, 3} and {5, 4}. The edge with minimum weight is edge {2, 8} which has weight 2. So include this edge and the vertex 8 in the MST.



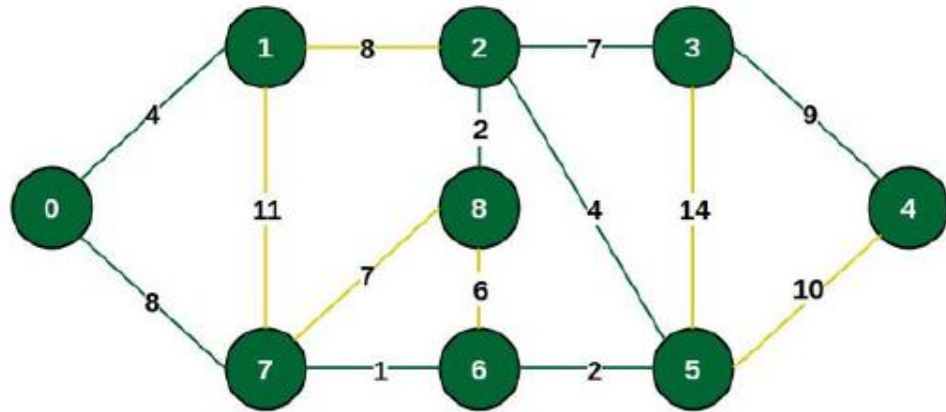
Prim's Algorithm ctd..

- Step 8: See here that the edges $\{7, 8\}$ and $\{2, 3\}$ both have **same weight which are minimum**. But **7 is already part of MST**. So we will consider the **edge $\{2, 3\}$** and include that edge and vertex 3 in the MST.



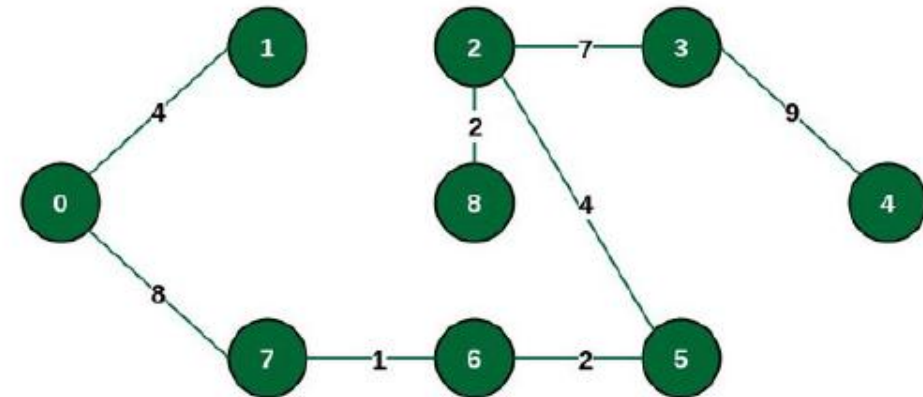
Prim's Algorithm ctd..

- Step 9: Only the **vertex 4** remains to be included. The minimum weighted edge from the incomplete MST to **4** is **{3, 4}**.



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

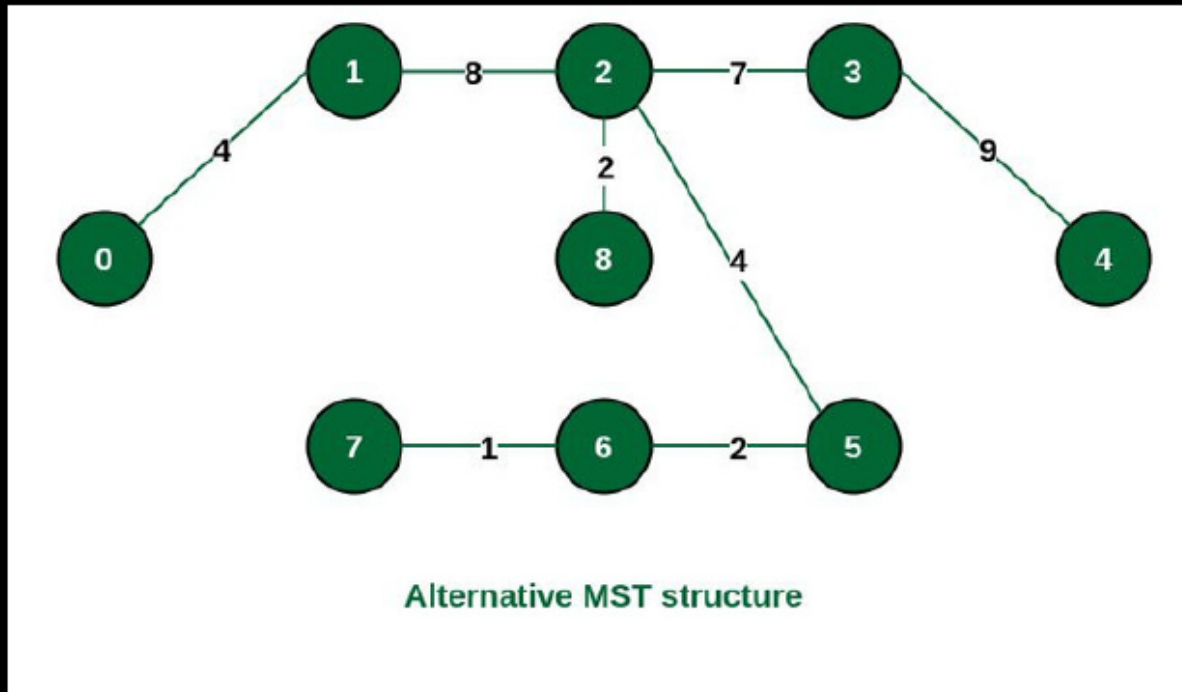
- The **final structure of the MST** is as follows and the weight of the edges of the MST is $(4 + 8 + 1 + 2 + 4 + 2 + 7 + 9) = 37$.



The final structure of MST

Prim's Algorithm ctd..

- Note: If we had selected the edge {1, 2} in the third step then the MST would look like the following.



Implement Prim's Algorithm

1. Choose any vertex as the starting vertex.

2. Initialize:

- *mstSet* = empty (stores vertices included in the MST)
- *key*[] = ∞ for all vertices, except the starting vertex (key = 0)
- *parent*[] = stores the parent of each vertex in the MST

3. Select the vertex *u* not in *mstSet* with the smallest key value.

4. Add *u* to *mstSet*.

5. Update the key values of all adjacent vertices of *u*:

- If an adjacent vertex is NOT in *mstSet* and the edge weight is less than its current key value, update:
 - *key*[*v*] = *weight*(*u*, *v*)
 - *parent*[*v*] = *u*

6. Repeat Steps 3–5 until all vertices are included in the MST.

7. Construct the MST using the *parent*[] array.

***The key is the minimum edge weight needed to connect a vertex to the growing Minimum Spanning Tree (MST).

Prims Algorithm

```
Prim(G):  
    Initialize MST = empty set  
    Select any starting vertex s  
    For each vertex v:  
        key[v] =  $\infty$   
        parent[v] = NULL  
    key[s] = 0  
    Repeat until all vertices are included:  
        Select vertex u with minimum key value  
        Add u to MST  
        For each adjacent vertex v of u:  
            If v is not in MST and weight(u,v) < key[v]:  
                key[v] = weight(u,v)  
                parent[v] = u  
    Return MST
```

Analysis of Prim's Algorithm

- Time Complexity:

- $O(V^2)$ using an **adjacency matrix** (or **simple array** implementation).
- $O(E \log V)$ using an **adjacency list** with a binary min-heap (**priority queue**).

- Space Complexity: $O(V)$

Where:

- V = Number of vertices
- E = Number of edges

Complexity of Prim's algorithm

The running time of the prim's algorithm depends upon using the data structure for the graph and the ordering of edges. Below table shows some choices -

Time Complexity

Data structure used for the minimum edge weight	Time Complexity
Adjacency matrix, linear searching	$O(V ^2)$
Adjacency list and binary heap	$O(E \log V)$
Adjacency list and Fibonacci heap	$O(E + V \log V)$

Prim's Algorithm

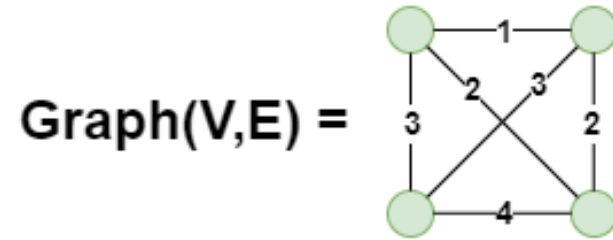
Advantages of Prim's Algorithm

1. **Guarantees** the Minimum Spanning Tree (**MST**) for a **connected, weighted graph**.
2. **Efficient** with $O(E \log V)$ time complexity using a binary heap (priority queue).
3. **Simple** and easy to understand and implement.
4. Performs **well on dense graphs** when implemented with an **adjacency matrix**.

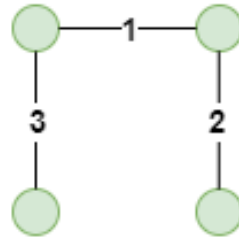
Disadvantages of Prim's Algorithm

1. Can be **slower on very large or dense graphs** due to processing many edges.
2. Requires a priority **queue (min-heap)**, which **uses additional memory**.
3. The choice of starting vertex can produce a **different MST** when multiple MSTs with the same total weight exist (although the **total cost remains the same**).
4. Works **only on connected graphs**;
5. For **disconnected** graphs, it produces a minimum spanning forest instead. union of the individual trees is called the Minimum Spanning Forest (**MSF**). The union of the individual trees is called the Minimum Spanning Forest (MSF).

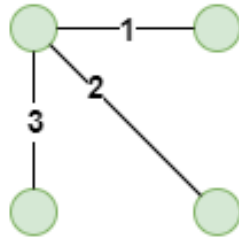
Multiple MSTs possible for same Graph



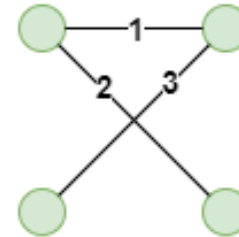
All Possible MST's of the above Graph



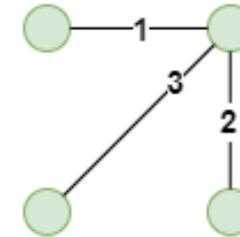
MST Cost =6



MST Cost =6



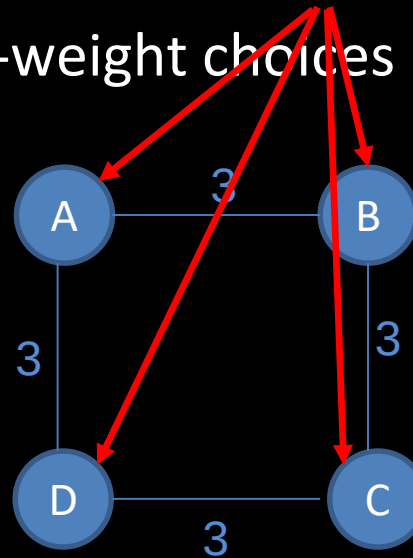
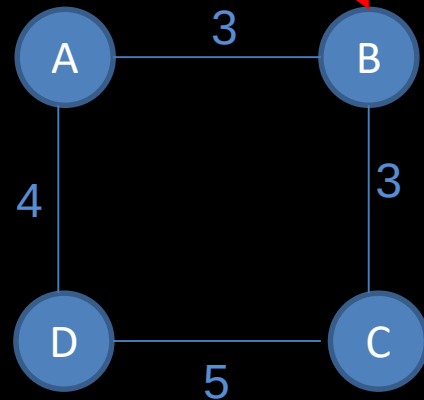
MST Cost =6



MST Cost =6

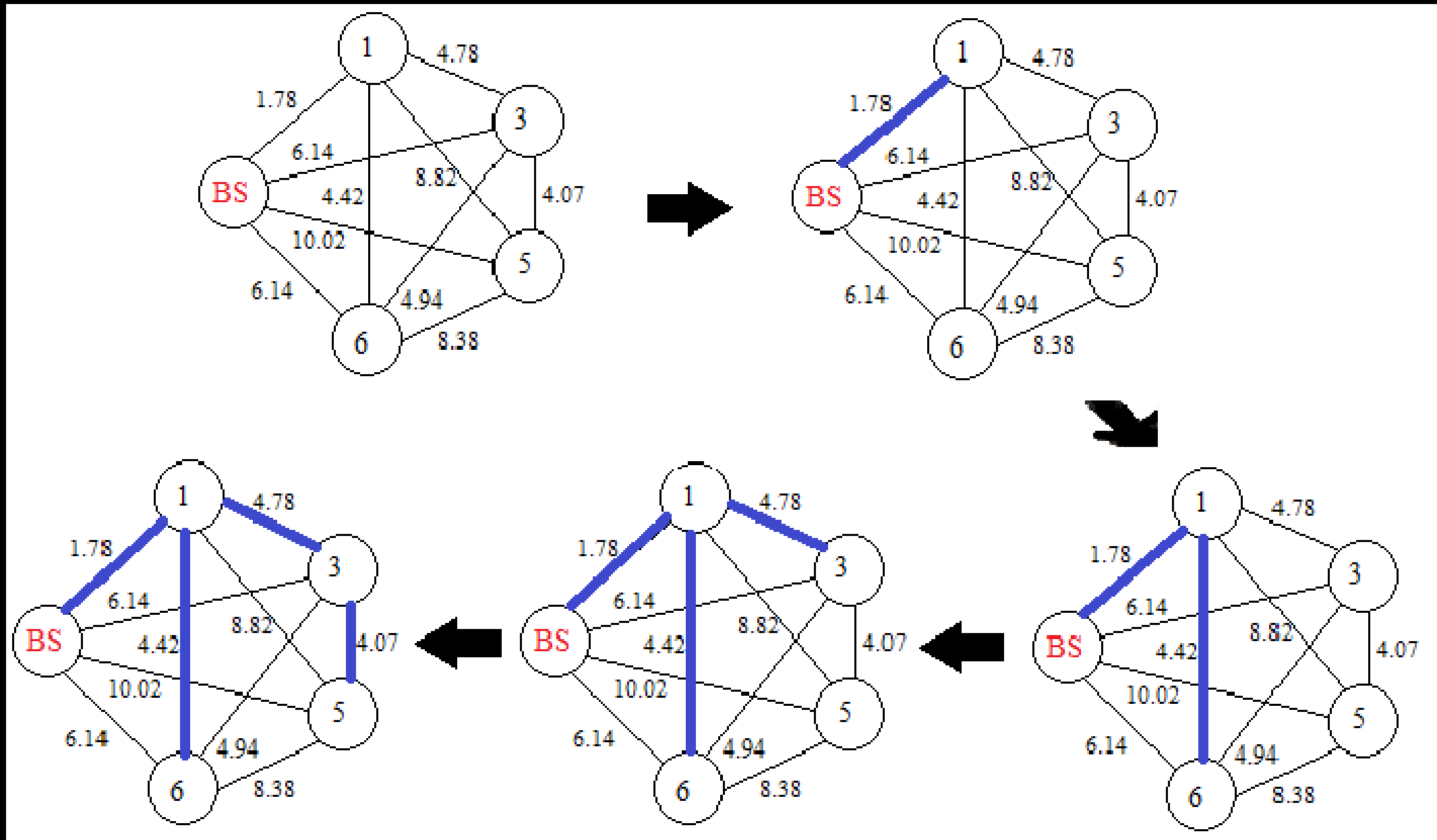
Multiple MSTs Reasons – Prim's Algorithm

1. MSTs can be possible in Prim's algorithm because the **starting node** can be chosen **arbitrarily**, **AND when equal-weight edges exist**, different choices can lead to different MSTs.
2. The **Starting node itself does NOT always create multiple MSTs**.
3. The **main reason is ties** (same minimum edge weights) during selection.
4. The **starting node** in Prim's algorithm can **SOMETIMES** result in different MSTs, but **only when** the graph has equal-weight choices (**ties**).



Multiple MSTs Reasons – Kruskal's Algorithm

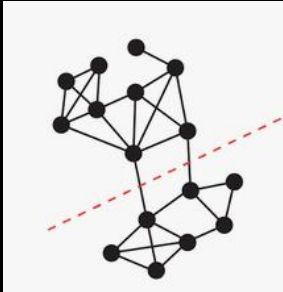
1. The algorithm does NOT choose a starting node. However, it always **sorts all edges by increasing weight** and selects the **smallest available edge** that **does not form a cycle**.
2. Multiple MSTs occur when there are **equal-weight edges AND different choices are possible among those edges**.



Kruskal's vs Prim's Algorithm

- Kruskal's Algorithm: Selects the **minimum-cost edge** first, **NOT a vertex**. It repeatedly chooses the **smallest-weight edge** that does NOT form a cycle.
- Prim's Algorithm: Starts from **any vertex**, then repeatedly selects **the minimum-cost edge that connects the current MST to a new vertex**.

Both are greedy algorithms that **rely on the Cut Property**, at each step, they pick the **minimum weight edge crossing a cut in the graph**, guaranteeing an optimal **MST**.



Kruskal's Algorithm considers in **terms of edges** (sort all edges, add smallest safe one), while Prim's Algorithm considers **in terms of vertices** (grow the tree one vertex at a time from a starting point).